AI Plant Diagnostic

By:

Sarim Amjad Nadri

39978

Abdullah Shahid

39260

Muhammad Sami Abbasi

46419

Supervised by:

Muhammad Waqar Arshad

Faculty of Computing

Riphah International University, Islamabad

Fall 2025

A Dissertation Submitted To

Faculty of Computing,
Riphah International University, Islamabad
As a Partial Fulfillment of the Requirement for the Award of the Degree of
Bachelors of Science in Computer Science

Faculty of Computing

Riphah International University, Islamabad

Date: 26/11/2025

Final Approval

This is to certify that we have read the report submitted by ***Sarim Amjad (39978)***, ***Abdullah Shahid (39260) and Sami Abbasi (46419)*** for the partial fulfillment of the requirements for the degree of the Bachelors of Science in Computer Science (BSCS). It is our judgment that this report is of sufficient standard to warrant its acceptance by Riphah International University, Islamabad for the degree of Bachelors of Science in Computer Science (BSCS).

Committee:

1

_____

Muhammad Waqar Arshad
(Supervisor)

2

_____

Dr. Musharaf Ahmed
(Head of Department/chairman)

Declaration

We hereby declare that this document "**AI Plant Diagnostic**" neither as a whole nor as a part has been copied out from any source. It is further declared that we have done this project with the accompanied report entirely on the basis of our personal efforts, under the proficient guidance of our teachers, especially our supervisor **Muhammad Waqar Arshad**. If any part of the system is proved to be copied out from any source or found to be reproduction of any project from anywhere else, we shall stand by the consequences.

Sarim Amjad

39978

Sami Abbasi

46419

Abdullah Shahid

39260

Dedication

We dedicate this project to our families for their unwavering support and encouragement throughout our academic journey. To our parents, thank you for believing in us and providing the love and motivation needed to pursue our dreams. Your sacrifices and endless patience have made this achievement possible. We also extend our heartfelt gratitude to our friends for their constant support and for always being there to lend a helping hand or a listening ear. Your camaraderie has made the challenges of this project more manageable and the successes even sweeter. A special thank you to our supervisor, **Sir Waqar Arshad,** for his invaluable guidance, insightful feedback, and continuous support. Your expertise and dedication have been instrumental in shaping this project and helping us overcome obstacles along the way. Lastly, we dedicate this work to anyone striving to improve agriculture quality. It is our hope that this project will contribute positively to the field of agriculture and assist those in need of reliable plant disease classification and care. Thank you all for being a part of this journey.

Acknowledgement

First of all, we are obliged to Allah Almighty the Merciful, the Beneficent and the source of all Knowledge, for granting us the courage and knowledge to complete this Project.

We would like to express our deepest gratitude to our supervisor, Sir Waqar Arshad and co supervisor Sir Hassan Ashraf, for his unwavering support, continuous guidance, and insightful feedback throughout the development of this project. His expertise and dedication have been instrumental in shaping our work and helping us navigate through various challenges. We are also immensely grateful to our families for their unwavering support and understanding during our academic journey. Your love and encouragement have been our driving force and have provided us with the strength to persevere. Additionally, we extend our appreciation to our friends and classmates for their assistance, moral support, and for creating a collaborative and motivating environment that made this project more manageable and enjoyable. Lastly, we acknowledge and thank everyone else who contributed to this project in any way. Your support and contributions have been greatly appreciated. Thank you all for being a part of this journey

Sarim Amjad
39978

Sami Abbasi
46419

Abdullah Shahid
39260

Table of Contents

List of Tables

List of Figures

Abstract

Agriculture remains the backbone of global food production, yet plant diseases continue to be one of the most significant challenges impacting crop quality, productivity, and farmer livelihoods. Millions of farmers worldwide lack timely access to agricultural experts who can diagnose plant diseases accurately. Traditional diagnosis methods rely on visual inspection, which is often unreliable due to the close resemblance between symptoms of various plant diseases. These limitations result in delayed or incorrect treatment, leading to unnecessary pesticide usage, diminished yields, and widespread economic losses. Consequently, there is a compelling need for an accessible, accurate, and technology-driven solution that empowers farmers to detect plant diseases early.

This project presents AI Plant Diagnosis, an innovative mobile application designed to enhance plant health monitoring through the integration of advanced deep learning techniques. The system focuses on image-based plant disease classification, enabling users to simply capture or upload an image of a plant leaf and receive an immediate diagnosis.

The application is developed using Flutter, offering a highly responsive, intuitive, and visually appealing interface compatible with Android Platform . The mobile interface is optimized for simplicity, enabling farmers, students, and agricultural stakeholders to use the system without requiring technical knowledge. Alongside the AI module, the system integrates a robust and secure backend responsible for managing user interactions, storing image data, processing classification results, and ensuring real-time communication between the app and the AI model. Strong data security practices and efficient database design guarantee user privacy, reliability, and seamless application performance.

AI Plant Diagnosis not only provides accurate disease classification but also enhances awareness by presenting essential information and general guidance related to the diagnosed disease. This allows users to take prompt and informed actions to protect plant health. As a result, the system helps minimize crop losses, reduce the misuse of chemical treatments, and promote healthier and more sustainable agricultural practices.

The successful development of AI Plant Diagnosis highlights the transformative potential of combining machine learning, agriculture, and mobile technology. By making advanced plant

disease classification accessible directly from a smartphone, the project offers a scalable and practical solution to a critical global challenge. Ultimately, this system empowers farmers with timely and reliable diagnostic assistance, supports informed decision-making, and contributes meaningfully to improving agricultural productivity and sustainability.

Introduction

1.1 Background

Agriculture plays a vital role in sustaining global food production and supporting the economic stability of millions of people, particularly in rural areas. However, plant diseases continue to present a major challenge to agricultural productivity worldwide. These diseases account for significant losses in crop yield and quality every year, threatening food security and creating financial difficulties for farmers. Despite the high prevalence of plant diseases, a large number of farmers fail to seek timely professional assistance due to several limiting factors. Common barriers include limited access to agricultural experts, long distances to agronomy centers, high consultation costs, inadequate awareness, and the difficulty of identifying plant diseases based solely on visual symptoms. As a result, many plant health issues remain untreated or mismanaged, leading to worsening disease severity and long-term economic losses.

Accurate classification of plant diseases is essential for effective crop management. However, traditional diagnostic practices face numerous obstacles. Many plant diseases exhibit highly similar visual patterns, such as leaf spots, blight, or discoloration, making it challenging even for experienced individuals to distinguish among them. Misdiagnosis often results in the use of incorrect pesticides or delayed action, which can further harm the plants and negatively impact crop yield. Additionally, professional agricultural services are often limited in remote farming communities, leaving farmers dependent on guesswork or unverified sources of information. These limitations highlight the urgent need for innovative, accessible, and reliable solutions that can assist farmers in accurately identifying plant diseases at an early stage.

Recent advancements in artificial intelligence (AI) and machine learning (ML) have paved the way for modernizing agricultural disease diagnosis. Image-based classification models, particularly those employing deep learning techniques, have shown remarkable accuracy in identifying plant diseases from leaf images.The rapid growth of smartphone usage worldwide provides an additional opportunity to integrate AI-driven diagnostic systems into practical agricultural tools. Mobile applications offer a convenient, accessible platform that allows farmers to capture or upload images and receive instant disease predictions without the need for specialized equipment or in-person expert consultations.

In response to these challenges, this project introduces AI Plant Diagnosis, a mobile-based application designed to support accurate and early detection of plant diseases through advanced ensemble learning techniques. The system integrates a powerful classification model with a

user-friendly mobile interface to help farmers identify diseases quickly and take timely action. By combining machine learning technologies with accessible mobile design, AI Plant Diagnosis aims to reduce crop losses, improve farm productivity, and promote more sustainable agricultural practices.

1.2 Goals and Objectives

1.2.1 Goals

- **Develop an Accessible Mobile Application:** Create a user-friendly and intuitive mobile platform using Flutter that allows farmers to quickly and accurately diagnose plant diseases through image uploads, ensuring ease of use even for users with limited digital literacy or in remote areas.
- **Integrate Advanced Machine Learning Techniques:** Implement a powerful deep learning model leveraging Convolutional Neural Networks (CNNs) for precise multi-class classification of plant diseases across various crop species.
- **Improve Farmer Engagement and Community Support:** Establish interactive features such as a community forum to promote active engagement, knowledge sharing among users.
- **Administration and Data Management**: Provide robust administrative functionalities through a dedicated Admin dashboard, enabling secure user management, oversight of community activities, and maintenance of data security and privacy.

1.2.2 Objective

- **Platform Development:** To design and develop a robust mobile application platform that includes standard user interface modules such as a home screen, a Diagnose module for image-based disease classification, a Community forum for user discussions, an Admin panel for management tasks. This objective emphasizes an intuitive user interface design and seamless navigation to facilitate user engagement and ease of use. Additionally, the platform will prioritize reliable performance and maintainability to support widespread adoption among the target user base.
- **Machine Learning Implementation:** To develop and train a Convolutional Neural Network (CNN) model for accurate classification of a broad range of plant diseases from images of affected leaves uploaded by users.
- **Improve Accessibility:** To enhance the accessibility and usability of the system for rural farmers by incorporating features and support channels tailored to their needs.

These include a community discussion forum that enables farmers to share experiences and seek peer advice, an integrated chat support feature that connects users with agricultural experts for real-time guidance, and a simplified navigation scheme with a clear interface optimized for individuals with limited technical literacy.

- **User Role Management:** To implement a robust role-based access control system that differentiates features and privileges for various user types, particularly user and Administrator, through appropriate access controls and tailored interfaces. For the user role, the system will provide a homescreen with access to the plant disease diagnostic tool, community discussion features. For the Administrator role, the system will offer an administrative dashboard equipped with tools to manage user accounts.

1.3 Scope of the Project

The scope of this project encompasses the design, development, and implementation of the AI Plant Diagnostic mobile application, integrating a CNN model for plant disease classification. The project includes the following key components:

- **Mobile UI & Navigation:** The app features a clear, streamlined interface (built in Flutter) so users can easily switch between sections. Primary screens include Diagnose (take/upload leaf photos for instant AI analysis). Community (in-app chat forums for farmers to share tips and alerts).
- **Machine Learning Integration:** To develop and train a Convolutional Neural Network (CNN) model for accurate classification of a broad range of plant diseases from images of affected leaves uploaded by users.
- **User Authentication & Roles:** The system implements secure login for user and Admin. Robust authentication is enforced by the backend. Farmers access the diagnostic tools, community chat. Admins have a separate dashboard to manage user accounts and update app resources.
- **Communication & Expert Support:** A built-in Community Chat connects users in a forum-like interface. Here users can discuss findings, share photos, and share alerts to eachother. For additional help, the app can offer expert consultation options, user may schedule a chat session with agronomy experts. These communication tools help users collaborate and seek personalized guidance .

Chapter 2: Literature Review

2.1 Introduction

Plant diseases are a major threat to global agriculture, dramatically reducing crop yields and compromising food security. Accurate and timely detection of plant diseases is therefore critical to mitigate economic losses and ensure crop health. However, traditional plant pathology practices rely on manual visual inspection by farmers or experts, which is time-consuming, labor-intensive, and prone to human error. In many cases, pathogens such as fungi, bacteria, and viruses can infect multiple parts of a plant making accurate diagnosis more complex.

Convolutional Neural Networks (CNNs) have shown remarkable performance in image-based plant disease classification. These deep learning models can automatically learn discriminative features and have been shown to capture subtle disease symptoms that might be overlooked by human observers. As a result, automated plant disease classification using AI has become a major research focus, with studies reporting faster and more consistent diagnoses compared to traditional methods.

This chapter provides a comprehensive review of existing literature on plant disease classification and diagnosis using artificial intelligence, particularly deep learning techniques. It encompasses an in-depth analysis of relevant research studies, algorithms, and methodologies employed by previous applications in this domain.

The literature review examines the definitions and descriptions of key concepts and approaches related to plant disease detection. It explores the various techniques and methods employed by different inventors and researchers. Each reviewed work is accompanied by its corresponding name, reference, inventor, and year of publication.

A summary table facilitates a comparative analysis of the reviewed works. The table presents a systematic overview of the literature, including the input and output of each approach, as well as a concise description of the method and its findings.

Furthermore, this chapter identifies the research gap and highlights the problem statement that the current project aims to address. This sets the foundation for the subsequent chapters, which will delve into the development and implementation of AI Plant Diagnostic, leveraging the insights gained from the literature review.

Overall, this literature review establishes a solid understanding of the existing knowledge and research landscape in plant disease detection and classification, providing a valuable framework for developing and evaluating AI Plant Diagnostic.

2.2 Background and Problem Elaboration

The health of global agriculture hinges directly upon the ability to conduct accurate and timely diagnosis of plant diseases. This necessity is paramount because plant pathologies and pests represent one of the most substantial threats to both food security and the economic viability of farm productivity. The extent of this challenge is staggering: conservative estimates indicate that diseases in plants are responsible for the destruction of approximately 20–40% of the global crop yield. Historic calamities, such as the infamous Irish Potato Famine, serve as stark, powerful reminders of the catastrophic social and economic consequences that result from the unchecked proliferation of virulent pathogens. This pervasive and urgent threat places an enormous premium on the development and widespread adoption of highly reliable and rapid classification systems for plant diseases.

Historically, the identification of plant ailments has been predominantly reliant on a combination of manual inspection and laboratory-based analytical methods. Field scouting and visual assessment, while a first line of defense, are fundamentally constrained. They are often time-consuming and labor-intensive, especially when deployed across vast cultivated areas. Crucially, the outcome of visual assessment is highly subjective, varying significantly based on the expertise and circumstances of the individual performing the inspection. More rigorous, conventional diagnostics, such as pathogen culturing, serological tests, or molecular assays, demand both significant time for processing and access to specialized equipment. These intrinsic limitations frequently lead to an unacceptable delay in intervention, allowing the disease to spread unchecked throughout the crop and ultimately maximizing the potential for severe yield losses before effective control measures can be deployed.

The advent of machine learning (ML) and image analysis promises a transformation in plant disease classification, offering a more objective and scalable solution. However, several practical challenges significantly impede the transition from promising research to reliable, real-world application on the farm. A major technical hurdle is the presence of environmental noise within field images. Factors like background clutter (e.g., weeds, soil), variable lighting conditions, the presence of multiple leaves, and physical occlusions (overlapping leaves) all

introduce noise that severely compromises the reliability and accuracy of classifier systems. Accurately training a robust ML model requires a foundation of diverse, high-quality data.

To overcome the challenges in current plant disease classification, particularly the need for robust multi-class identification, our work proposes a novel approach that moves beyond fragmented tools. Currently, no single, comprehensive digital platform exists that efficiently handles the diverse range of crop diseases while simultaneously integrating human expertise. Therefore, a core component of our solution is the development of a unified system that not only utilizes advanced AI-driven classification but also crucially incorporates direct communication features. This platform would establish vital channels for expert consultation (doctor-to-user chat) and community interaction (user-to-user chat). This dual-layer approach provides a crucial mechanism for validating machine predictions, enabling practitioners to quickly seek advice from pathologists, share localized disease knowledge, and transform the isolated task of diagnosis into a collaborative and supported process.

2.3 Detailed Literature Review

2.3.1 Definitions

- **Plant Disease Classification:** Automated identification of diseases from plant leaf images using AI models, with a focus on early and accurate recognition to improve yield and reduce losses.
- **Convolutional Neural Networks (CNNs):** A type of deep learning model specifically suited for image analysis. CNNs extract hierarchical features and patterns that are essential in identifying complex disease symptoms in plant leaves.
- **Vision Transformers (ViTs):** Deep learning models that apply self-attention mechanisms on image patches. They enable effective global context modeling, often surpassing CNNs in large-scale image classification tasks.
- **Hybrid Deep Learning Models:** These models combine the strengths of multiple architectures such as CNN with ViT or CNN with thermal imaging to enhance classification accuracy and robustness in variable field conditions.

2.3.2 Related Research Work 1

[1] proposed a novel deep learning model that integrates simulated thermal imaging with conventional RGB inputs for improved diagnosis of paddy leaf diseases. The motivation stems

from the observation that early-stage leaf infections often present subtle physiological changes such as slight shifts in transpiration or chlorophyll content that are not easily visible in standard RGB imagery. By simulating thermal channels and incorporating them into the training pipeline, the study aimed to improve detection of such early indicators and enhance model generalizability under diverse environmental conditions.

The architecture was based on two pre-trained CNN models: DenseNet121 and MobileNetV2, both known for their strong feature extraction capabilities and relatively low computational cost. The authors created a dual-path hybrid model wherein RGB and simulated thermal images were processed in parallel, followed by feature fusion through concatenation and fully connected layers. Data augmentation techniques such as rotation, shearing, contrast stretching, and noise injection were extensively applied to both image types. This ensured that the model was robust to variations in leaf orientation, brightness, and real-world imaging artifacts.

A custom dataset of paddy leaf images was enhanced using a simulation pipeline that generated pseudo-thermal maps. These maps mimicked heat signatures indicative of water loss or cell degradation, often associated with fungal or bacterial infections. The model was trained using categorical cross-entropy loss and optimized via the Adam optimizer with an adaptive learning rate scheduler. Evaluation was conducted on a stratified test set, measuring accuracy, precision, recall, F1-score, and Area Under the Curve (AUC).

The results were compelling: the RGB-only baseline achieved 94.2% accuracy, whereas the hybrid model incorporating simulated thermal features reached 97.9% accuracy and an AUC of 98.6%. Class-wise performance showed especially high sensitivity in detecting diseases like bacterial leaf blight and brown spot—both known for their subtle symptomatology. In addition, the authors used Grad-CAM visualizations to verify that the thermal input stream allowed the model to focus on biologically relevant heat zones rather than background or lighting artifacts.

While the study demonstrates significant advancements, it also acknowledges limitations. The simulated thermal generation process, while innovative, may not perfectly replicate real thermographic imaging. Furthermore, training a hybrid model with dual-input streams increases computational load and training time, making deployment on resource-constrained mobile platforms challenging without optimization techniques like model pruning or quantization. Nevertheless, the findings lay a solid foundation for future models that might integrate real thermal sensors into field-use diagnostic tools for rice crops, bridging a major technological gap in early disease detection.

Overall, [1] showcases how integrating multimodal imaging, even in a simulated form, can significantly enhance the diagnostic capabilities of deep learning models in agriculture. It

highlights the growing importance of not just deeper architectures, but smarter, biology-aware input designs in building more effective plant disease classification systems.

2.3.3 Related Research Work 2

[2]developed a robust plant disease recognition framework combining Convolutional Neural Networks (CNNs) with the YOLOv7 object detection model to identify and localize multiple tomato plant diseases. This hybrid approach is designed to provide both classification and spatial mapping, addressing a critical need in field applications where understanding the affected area is just as important as identifying the disease type.

The framework was built using a custom-curated dataset of tomato leaf images infected with early blight, mosaic virus, septoria leaf spot, and bacterial spot. The dataset included high-resolution images captured under varied lighting and environmental conditions to simulate real-world scenarios. For the classification task, a CNN model was trained using transfer learning with ResNet50 as the base, while YOLOv7 was integrated to perform bounding-box-based object detection.

Preprocessing steps included resizing images to 256x256 pixels, applying CLAHE for contrast enhancement, and performing mean normalization. Data augmentation techniques such as horizontal flipping, rotation, and brightness adjustment were employed to improve model robustness. During training, the CNN component handled multi-class classification, while the YOLOv7 stream simultaneously learned to detect and outline infected regions on the leaf.

Performance evaluation revealed impressive results. The CNN classifier achieved 94.4% accuracy, while the integrated YOLOv7 detection module recorded a mean average precision (mAP) of 96.83%. The combined system allowed for dual-level decision-making, where the model not only predicted the disease class but also highlighted specific symptomatic zones. This is particularly useful in mobile and drone-based deployments where users require both identification and visual confirmation.

The model's strength lies in its holistic understanding of image data combining global classification and localized detection. However, [2] also acknowledged limitations, particularly in terms of computational complexity. Running YOLOv7 alongside a CNN classifier increases memory usage and inference time, which may affect real-time performance on lower-end mobile devices. To mitigate this, the authors suggest future enhancements such as model pruning, quantization, or distillation.

The study concludes with practical implications, emphasizing that this hybrid architecture could be effectively deployed in mobile apps to assist farmers in diagnosing tomato plant diseases in real time. The spatial outputs from YOLOv7 further support educational and advisory applications by visually illustrating disease spread. The dual-function design exemplifies a trend in precision agriculture merging classification and localization into a unified AI-driven decision-support system.

2.3.4 Related Research Work 3

[3] introduced a hybrid deep learning architecture that combines the structural strengths of Inception and Xception CNN models to improve the multi-class classification of plant leaf diseases. The proposed system addresses two major bottlenecks in plant disease classification: limited generalization capacity of individual CNNs and increasing model size that hampers deployment in mobile applications. By blending Inception's multiscale convolutional kernels and Xception's efficient depthwise separable convolutions, this hybrid network strikes a balance between feature richness and parameter economy.

The dataset used for evaluation was sourced from the publicly available Plant Village repository, comprising over 50,000 high-resolution images spanning 38 disease categories. Preprocessing steps included normalization, histogram equalization, and removal of background noise. To augment training diversity, the authors employed horizontal flipping, zooming, shearing, rotation, and Gaussian noise injection. This helped mitigate class imbalance and improve the model's resilience to real-world distortions.

The architecture begins with parallel convolutional modules from both Inception and Xception branches, which independently process the input before concatenation. This fused representation is passed through batch normalization, dropout, and fully connected layers. The final output layer uses SoftMax activation for multi-class disease classification. Training was carried out using the Adam optimizer and categorical cross-entropy loss. Regularization was achieved through dropout (set at 0.5) and early stopping.

The model achieved a top-1 classification accuracy of 98.7% and an F1-score of 0.987, outperforming baseline models like VGG16 (94.6%), ResNet50 (95.2%), and MobileNetV2 (93.8%). Moreover, inference tests on TensorFlow Lite demonstrated that the hybrid model performed efficiently on low-end devices without sacrificing prediction speed. The authors also provided Grad-CAM heatmaps to validate the interpretability of their predictions, confirming the model's focus on biologically meaningful leaf regions.

The study concludes that hybrid CNNs are not merely an academic pursuit but a practical necessity when deploying AI in real agricultural settings. By combining architecture-level innovations with model compression readiness, [3] opens avenues for embedding high-performance disease classifiers into handheld diagnostic tools.

2.3.5 Related Research Work 4

[4] proposed PWD ViT Net, a lightweight and efficient model that merges the local feature extraction capabilities of CNNs with the global contextual understanding of Vision Transformers (ViTs). The system was designed for the early detection of pine wilt disease, a critical threat to forestry health. The novelty lies in using a dual-stream encoder architecture that processes images via CNN and ViT branches simultaneously, fusing their outputs for final classification.

The dataset used in the study consisted of pine needle images annotated for early wilt symptoms. Preprocessing involved image resizing to 224×224, denoising using Gaussian filters, adaptive histogram equalization, and background masking. The CNN encoder extracted localized features such as lesion textures and needle curling, while the ViT module encoded spatial relationships across the image. Fusion occurred through a concatenation layer, followed by two dense layers and a SoftMax classifier.

Training was done using a learning rate scheduler and label smoothing to prevent overconfidence in class predictions. The total model size was kept below 8M parameters, making it suitable for mobile inference. Evaluation metrics included accuracy, AUC, and inference latency. The hybrid model achieved 96.3% accuracy and an AUC of 95.8%, outperforming both standalone CNN and ViT models.

A significant strength of PWDViTNet is its capacity for real-time analysis on edge devices like the Jetson Nano, opening possibilities for integration with UAVs for large-scale forest monitoring. However, [4] acknowledges that ViT-based models require larger datasets to reach optimal performance and suggests integrating attention refinement or token pruning in future iterations. Overall, PWDViTNet demonstrates how transformer-based architectures, when intelligently hybridized with CNNs, can serve practical needs in agricultural and environmental AI applications.

2.3.6 Related Research Work 5

[5]conducted a detailed comparative study evaluating the effectiveness of deep learning specifically CNNs against classical machine learning models like Random Forest (RF),

Support Vector Machines (SVM), and k-Nearest Neighbors (KNN) for identifying plant diseases from leaf images. The goal was to determine not only the accuracy gap between traditional ML and CNNs but also to assess robustness, scalability, and adaptability under noisy real-world conditions. The study used both the PlantVillage dataset and field-collected samples that introduced challenges like leaf occlusion, inconsistent lighting, and varied background noise.

The classical models relied on handcrafted features, such as color histograms in HSV space, texture descriptors using Gray-Level Co-occurrence Matrix (GLCM), and shape descriptors like edge contour mapping. These features were extracted and input into RF, SVM, and KNN classifiers, tuned via grid search and validated using k-fold cross-validation (k=5). In contrast, the CNN models (based on VGG19 and MobileNet) were trained end-to-end, with automatic feature extraction and fine-tuning using transfer learning. Data augmentation techniques like random rotation, zoom, and horizontal flipping were used to enrich the training set for CNNs but not for traditional ML models.

Evaluation across accuracy, precision, recall, F1-score, and robustness under noise showed that CNNs significantly outperformed traditional models. CNNs achieved an average classification accuracy of 98.2%, compared to 89.5% for RF, 85.3% for SVM, and 81.2% for KNN. CNNs also showed higher generalization ability when tested on unseen field images, maintaining consistent accuracy above 96%, whereas the traditional models dropped to as low as 70% due to their dependence on specific handcrafted features.

Another dimension evaluated was inference time and computational overhead. Classical models had faster training times and lower memory usage but required pre-engineered pipelines. CNNs, though heavier in computation, offered scalability and automatic learning of complex patterns, reducing dependency on domain expertise.

The authors concluded that while RF and SVM might still be useful for small-scale or low-resource applications, CNNs are vastly superior in terms of robustness, scalability, and real-time deployment readiness. The study's significance lies in its empirical evidence supporting the shift from manual feature engineering to automated deep learning for plant pathology.

2.3.7 Related Research Work 6

[6] proposed a novel deep learning architecture combining the strengths of a Variational Autoencoder (VAE) and a Vision Transformer (ViT) for enhanced plant disease detection. This approach is grounded in the idea that VAE can distill latent feature representations that preserve

semantic meaning, while ViT excels at capturing long-range dependencies across image patches. The dual-encoder system thus merges generative unsupervised learning with global attention-based classification for robust and accurate plant disease recognition.

The dataset used consisted of high-resolution leaf images from tomato, apple, and maize plants affected by common diseases such as blight, powdery mildew, mosaic virus, and rust. Preprocessing involved denoising, segmentation to isolate the leaf from the background, and normalization. The VAE module comprised an encoder-decoder pair that learned a compressed latent representation. These embeddings were then passed into a ViT model pre-trained on ImageNet and fine-tuned for the plant disease domain. The fusion of both outputs was achieved through concatenation and fed into a fully connected network for classification.

To enhance model generalization, extensive augmentation strategies were applied. These included elastic deformations, channel shuffling, and even domain transfer via style transfer from different plant species. Training was conducted using a combination of reconstruction loss (for VAE) and categorical cross-entropy (for ViT). The hybrid model was validated on a held-out test set and compared against baseline CNN and ViT models.

Results were impressive: the VAE-ViT model achieved a classification accuracy of 97.8% and an F1-score of 0.961. Notably, it demonstrated strong cross-crop generalization, maintaining above 94% accuracy even when tested on crops it was not explicitly trained on. The study also included ablation tests that confirmed the VAE component improved feature quality by reducing intra-class variability and increasing class separability.

Despite its strength, the model's limitation lies in training complexity and the requirement for high-end hardware due to ViT's large parameter count. The authors suggested lightweight token pruning and knowledge distillation as future work to enable mobile deployment.

This study stands out for its innovative combination of generative modeling and attention-based learning. It sets a precedent for future research aiming to create universal plant disease detection systems that can operate across species and environmental domains.

2.3.8 Related Research Work 7

[7]explored the use of transfer learning on Convolutional Neural Networks (CNNs) to create an accessible, real-time plant disease diagnosis application for mobile platforms. Their study addressed a major usability gap: the lack of lightweight, high-accuracy AI tools that can run offline on low-cost smartphones, especially for smallholder farmers in remote regions.

The researchers employed a subset of the PlantVillage dataset, comprising over 50,000 annotated images across 14 crop species and 26 disease classes. Images underwent standard

preprocessing steps including normalization, color histogram equalization, and noise filtering. Augmentation was applied to enrich the training dataset, using rotation, flipping, and brightness modulation to simulate varied field conditions.

For the model backbone, DenseNet121 and ResNet50 were selected due to their proven balance between accuracy and parameter efficiency. Transfer learning was applied by freezing the initial layers trained on ImageNet and retraining the final dense layers with agricultural image data. The models were trained using categorical cross-entropy loss and evaluated using standard metrics like accuracy, precision, recall, and F1-score.

The results were promising: DenseNet121 achieved a classification accuracy of 97.5%, outperforming ResNet50 by 1.2%. To prepare the model for mobile use, it was converted to TensorFlow Lite (TFLite) and optimized using quantization and pruning, which reduced model size by over 60% without notable loss in accuracy. On-device inference time was under 400ms on a mid-range Android phone.

The mobile app itself was developed using the Flutter framework, ensuring compatibility with both Android and iOS. Its interface allowed users to capture or upload leaf images, receive instant disease classification results, and access treatment guidance and symptom images. A key innovation was its offline functionality, enabling full diagnosis capabilities without internet access a critical feature for rural usage.

The study also conducted field trials involving local farmers, demonstrating practical use and gathering feedback to improve user interface and confidence in AI predictions. Limitations noted included occasional misclassification when leaf backgrounds were cluttered or when lighting conditions were extreme, suggesting future work could explore additional filters or background segmentation modules.

[7]stands out by demonstrating a complete, end-to-end deployment pipeline from data preparation and model training to app design and field usability testing. It proves that CNN-based systems, when paired with thoughtful engineering and real-user focus, can bridge the gap between cutting-edge AI and grassroots agricultural support.

2.4 Literature Review Summary Table

Table 1: Summary of Plant Disease Classification using Deep Learning

No	Reference	Title	Year	Model	Dataset	Results
1	[1]	Enhancing Paddy Leaf Disease Diagnosis using Hybrid CNN with	2025	CNN + Thermal	Paddy Leaf (Simulated)	Accuracy: 97.9%, AUC: 98.6%

		Simulated Thermal Imaging				
2	[2]	Tomato Disease Recognition using CNN and YOLOv7	2024	CNN + YOLOv7	Custom Tomato Leaf	Accuracy: 94.4%, mAP: 96.83%
3	[3]	Hybrid Inception-Xception CNN for Plant Disease Classification	2025	Hybrid CNN	PlantVillage	Accuracy: 98.7%, F1: 0.987
4	[4]	PWDViTNet: Early Pine Wilt Detection using CNN-ViT	2025	CNN + ViT	Pine Needle Dataset	Accuracy: 96.3%, AUC: 95.8%
5	[5]	Comparative Analysis of CNN vs RF/SVM/KNN for Plant Disease Detection	2024	CNN vs ML	PlantVillage + Field	CNN Accuracy: 98.2%, RF: 89.5%
6	[6]	Cross-Crop Disease Classification using VAE + ViT	2025	VAE + ViT	Tomato, Apple, Maize	Accuracy: 97.8%, F1: 0.961
7	[7]	Mobile Plant Disease Diagnosis using Transfer Learning	2024	DenseNet121 (TFLite)	PlantVillage	Accuracy: 97.5%, Inference: <400ms

2.5 Research Gap

Despite notable advancements in deep learning approaches for plant leaf disease classification, several critical research gaps persist that hinder the development of fully reliable and deployable systems. While many Convolutional Neural Network (CNN)-based models achieve high accuracy on benchmark datasets, they often fail to address the complex variability inherent in plant disease symptoms. This limitation becomes especially evident in scenarios where different diseases exhibit overlapping visual manifestations. For instance, early blight and nutrient deficiencies may both present with yellowing, spotting, or marginal necrosis, making differentiation extremely challenging for models lacking fine-grained feature extraction. Such

overlap significantly increases the risk of misclassification in real-world applications, underscoring the need for architectures capable of capturing subtle inter-disease distinctions to ensure accurate and timely diagnoses.

Another prominent research gap concerns the quality and composition of existing plant disease datasets. Many publicly available datasets are captured under controlled laboratory conditions, with uniform lighting, clean backgrounds, and single leaves placed against plain surfaces. Although beneficial for initial model development, these datasets fail to represent the complexity of real agricultural environments, where leaves exhibit diverse orientations, occlusions, weather damage, pest interference, and variable lighting conditions. Background clutter such as soil, branches, shadows, and other leaves often distracts CNNs from learning salient disease features, leading to poor model generalization when deployed in fields.

In addition to environmental discrepancies, class imbalance remains a persistent challenge in plant disease datasets. Common diseases like leaf spot or blight are typically overrepresented, whereas rare but critical diseases such as viral infections or wilt pathogens occur with considerably fewer samples. This imbalance skews the learning process, causing models to become biased toward majority classes and demonstrating reduced sensitivity when identifying minority or early-stage diseases. Without addressing such imbalance through advanced sampling strategies or loss-function engineering, existing models struggle to maintain consistent diagnostic performance across all classes.

Dataset bias extends further into cultivar and species diversity. Many datasets inadequately represent the wide range of plant varieties, leaf shapes, colors, and growth stages found in agricultural settings. A model trained predominantly on one cultivar may perform poorly when exposed to another with visually distinct leaf morphology. Moreover, environmental and regional variations such as humidity, soil nutrients, or localized pathogens can alter disease presentations, making it difficult for models trained on limited data sources to generalize across global agricultural contexts. This lack of diversity raises concerns regarding the robustness and inclusiveness of current classification systems.

Furthermore, both intra-class and inter-class variability in plant disease symptoms pose substantial obstacles. Diseases may manifest differently depending on plant maturity, environmental stress, or infection severity, resulting in high intra-class variability. Conversely,

different diseases often exhibit highly similar symptoms, such as chlorosis, discoloration, or the presence of spots, leading to significant inter-class similarity. Standard CNN models with fixed kernel sizes and static feature hierarchies may fail to capture these complex and multi-scale characteristics, limiting their capability to accurately differentiate between visually correlated disease types. This architectural rigidity ultimately constrains diagnostic reliability.

Finally, the computational complexity and resource demands of state-of-the-art deep learning models present barriers to real-world deployment, particularly in low-resource agricultural regions. Deep architectures often suffer from vanishing or exploding gradients during training, and require substantial GPU resources for both training and inference. Such requirements restrict their accessibility for farmers, field technicians, and agricultural extension workers who rely on mobile or edge-based solutions. Without efficiency-focused models, the scalability of plant disease detection technologies remains limited, impeding their integration into large-scale agricultural systems.

In summary, although deep learning has significantly advanced multi-class plant leaf disease classification, addressing these research gaps is crucial for building robust, accurate, and field-ready diagnostic systems. Future research must emphasize improved model architectures capable of handling overlapping features, the development of realistic and diverse datasets, the mitigation of class imbalance and dataset bias, and the optimization of computational efficiency. Tackling these challenges will unlock the full potential of AI-driven plant disease diagnostics, resulting in improved crop health monitoring, increased agricultural productivity, and more sustainable farming practices.

2.6 Problem Statement

Plant diseases significantly undermine crop productivity and threaten the livelihoods of farmers, particularly those operating in rural areas where access to plant pathology expertise is limited or non-existent. Traditional diagnostic methods often require sending samples to distant laboratories or consulting scarce specialists, leading to delayed or inaccurate disease identification. Moreover, existing digital solutions frequently focus on single-disease classification or are geared toward well-resourced agricultural settings, leaving smallholder farmers and urban enthusiasts without a reliable, scalable diagnostic tool. As a result,

unaddressed infections can spread rapidly, reducing yields, increasing input costs, and exacerbating food security challenges in vulnerable communities.

The core challenge, therefore, is to develop an accessible platform that can simultaneously identify multiple diseases across various plant species using only widely available mobile devices. Such a solution must deliver rapid, accurate results without requiring specialized equipment or technical expertise. By enabling prompt, data-driven disease diagnosis, this platform will empower farmers and urban growers alike to make informed management decisions, minimize crop losses, and reduce dependency on scarce plant pathology resources.

Chapter 3: Requirements and Design

3.1 Requirements

3.1.1 Functional Requirements

1. **Upload Pic**
    - User uploads a plant image for disease detection. System should validate and pre-process the image. AI model predicts disease type.

2. Sign up / Login

- Secure user registration with email/password. Login (admin, client, pathologist).

3. View User (Admin)

- Admin can view details of registered users.

4. Block User (Admin)

- Admin can block abusive or suspicious users.

5. Generate Report

- Auto-generate disease diagnosis reports in PDF.

6. Manage Cart / Checkout

- Add/remove medicines from cart and place orders.

7. Message Doctor / Receive Message

- Two-way messaging between client and pathologist.

8. Chat in Community

- Clients and pathologists interact in public threads.

3.1.2 Hardware and Software Requirements

1. **Operating System**:

    - Windows 10 or above / Linux / macOS (for admin panel)
    - Android 12 or above (for client-side app)

2. **Backend**:

    - Node.js (API server)
    - Python (for AI/ML image analysis)

3. **Frontend**:
    - Flutter (for cross-platform mobile app)
4. **Database**:
    - MongoDB
5. **Image Processing & AI Tools**:
    - OpenCV, TensorFlow for disease classification
    - REST API for AI model integration
6. **Chat & Community Tools**:
    - Web Sockets for real-time messaging
    - Community forum integration
7. **Security Tools**:
    - JWT for authentication.
8. **Client/Doctor Devices**:
    - Android smartphones (with camera support)
    - Minimum 3 GB RAM for smooth app usage
9. **Admin Devices**:
    - PC/Laptop with at least 8 GB RAM
    - 2.4 GHz dual-core processor
    - Internet connection

3.2 Proposed Methodology

The proposed methodology for the AI Plant Diagnostic system integrates mobile-based image acquisition with backend AI-based disease classification. The development is divided into distinct stages to ensure modularity, scalability, and maintainability.

3.2.1 Data Preparation and Preprocessing

For the plant leaf disease classification task, we utilized publicly available datasets, specifically the Plant Village and Cassava Leaf Disease (CLD) benchmark datasets, which provide a rich set of images for model training and evaluation. These datasets cover a wide range of crops and diseases, offering good quality images for various plant species. By employing these datasets, we ensure that the model is exposed to a diverse array of plant diseases, facilitating robust training across multiple classes and improving the model's ability to generalize to unseen data.

3.2.2 AI Model Development

The proposed framework for plant leaf disease classification utilizes a hybrid CNN model that integrates ResNet-50 and InceptionV3 through feature-level fusion. The input image is processed by both networks to extract distinctive features, which are then aggregated using Global Average Pooling into 2048-dimensional vectors. These vectors are concatenated into a 4096-dimensional feature vector, allowing the model to leverage complementary information from both networks. The concatenated vector is passed through a linear classifier, followed by a Softmax layer to predict the disease class. This approach enhances classification accuracy and robustness, effectively handling the complexities of plant leaf disease identification and providing a reliable solution for real-time agricultural monitoring and intervention.

3.2.3 Model Deployment Strategy

Once trained and validated, the proposed CNN model was prepared for integration with the mobile application. Two primary deployment strategies were considered to ensure flexibility and performance:

1. **On-Device Deployment:** The model is converted into a TensorFlow Lite (.tflite) format. This lightweight version is embedded directly within the Flutter application, allowing for fast, offline inference without needing an internet connection.

3.2.4 Frontend Development with Flutter

The client-facing mobile application was developed using Flutter, Google's cross-platform UI toolkit. This choice enables the deployment of a single codebase to Android, significantly reducing development time and ensuring a consistent user experience. The application is modular and feature-rich, including:

- **Image Upload & Diagnosis:** The core feature allowing users to capture or upload a plant leaf image for analysis.
- **Community Forum:** A space for users to discuss results, share tips, and interact with experts.
- **Admin Dashboard:** A separate interface for administrators to manage users and oversee system activity.

3.2.5 Backend Architecture and Authentication

A robust backend was developed using **Node.js** to manage server-side logic, data, and user authentication. User security is handled through **JSON Web Tokens (JWT)**, which securely manage user sessions after they log in.

The data management strategy follows the follow approach:

- **MongoDB:** Serves as the primary persistent database for storing user account information, community chat and other critical data.

This architecture ensures a scalable and efficient system capable of handling real-time communication and long-term data storage.

3.2.6 System Testing and Evaluation

A comprehensive evaluation was conducted to validate both the AI model and the mobile application. The proposed CNN model's performance was assessed against a dedicated test dataset using standard classification metrics:

- **Accuracy:** The overall percentage of correct predictions.
- **Precision, Recall, and F1-Score:** Metrics used to evaluate the model's performance for each individual disease class.

3.3 System Architecture

The system architecture is designed using a layered and modular approach to ensure scalability, performance, and ease of maintenance.

3.3.1 Presentation Layer

- Patient (Mobile): upload leaf images for diagnosis, view diagnoses, participate in community chat, private chat with doctors, book/manage appointments; purchase medicines,
- Doctor (Mobile): secure login, schedule, review cases and validate AI results, private chat with patients, upload prescriptions/treatment plans in chat, manage appointments
- Admin (Web): upload product, manage doctor, block user

3.3.2 Database Design

Technology: MongoDB

Functionality:

- **Flexible Data Storage:** Hold user, doctor and admin information.
- **Document Database:** Stores all related information together in single, simple "documents **faster reading** and easier changes than a traditional table setup.

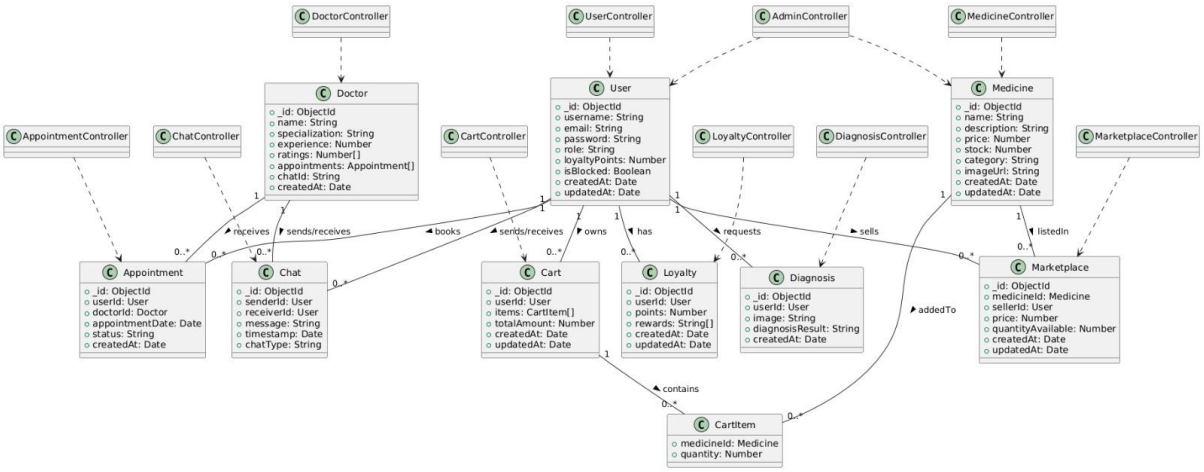

Figure 1:Database diagram

3.3.3 External integration layer

- Stripe payment

- Ensemble Model

3.3.4 Service Layer

- Chat Api
- Appointment Api
- Market place Api
- Auth Api

3.4 System Architecture diagram

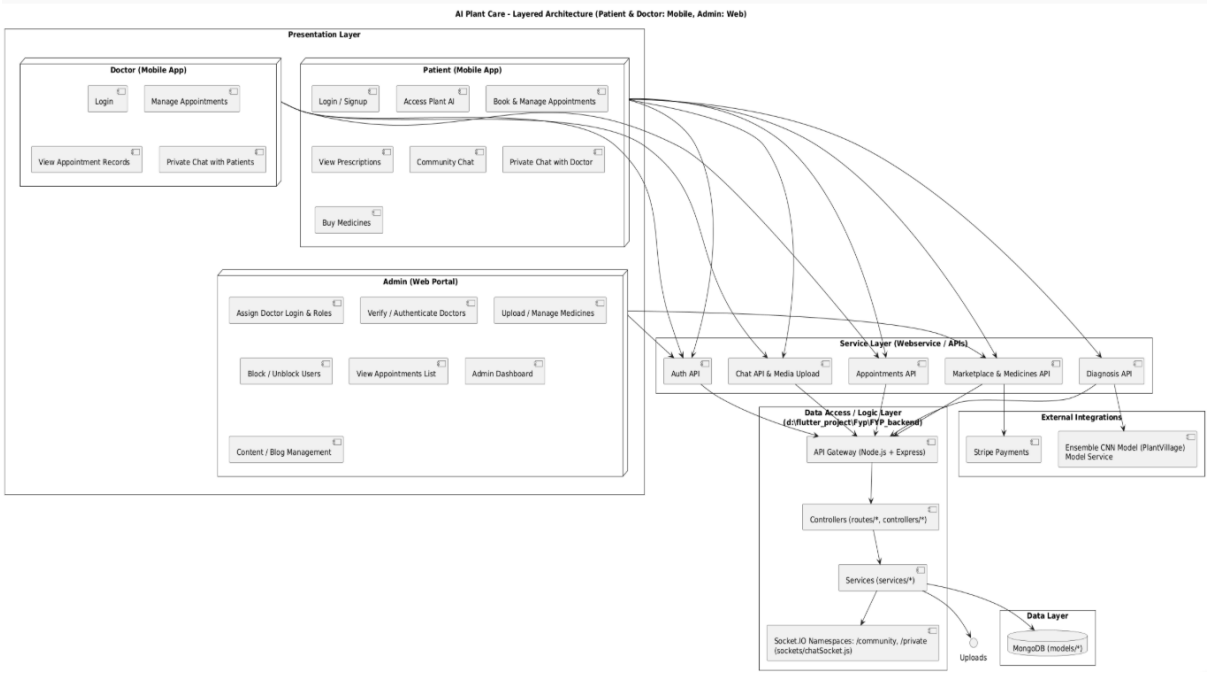

Figure 2:System Architecture diagram

3.5 Use Cases

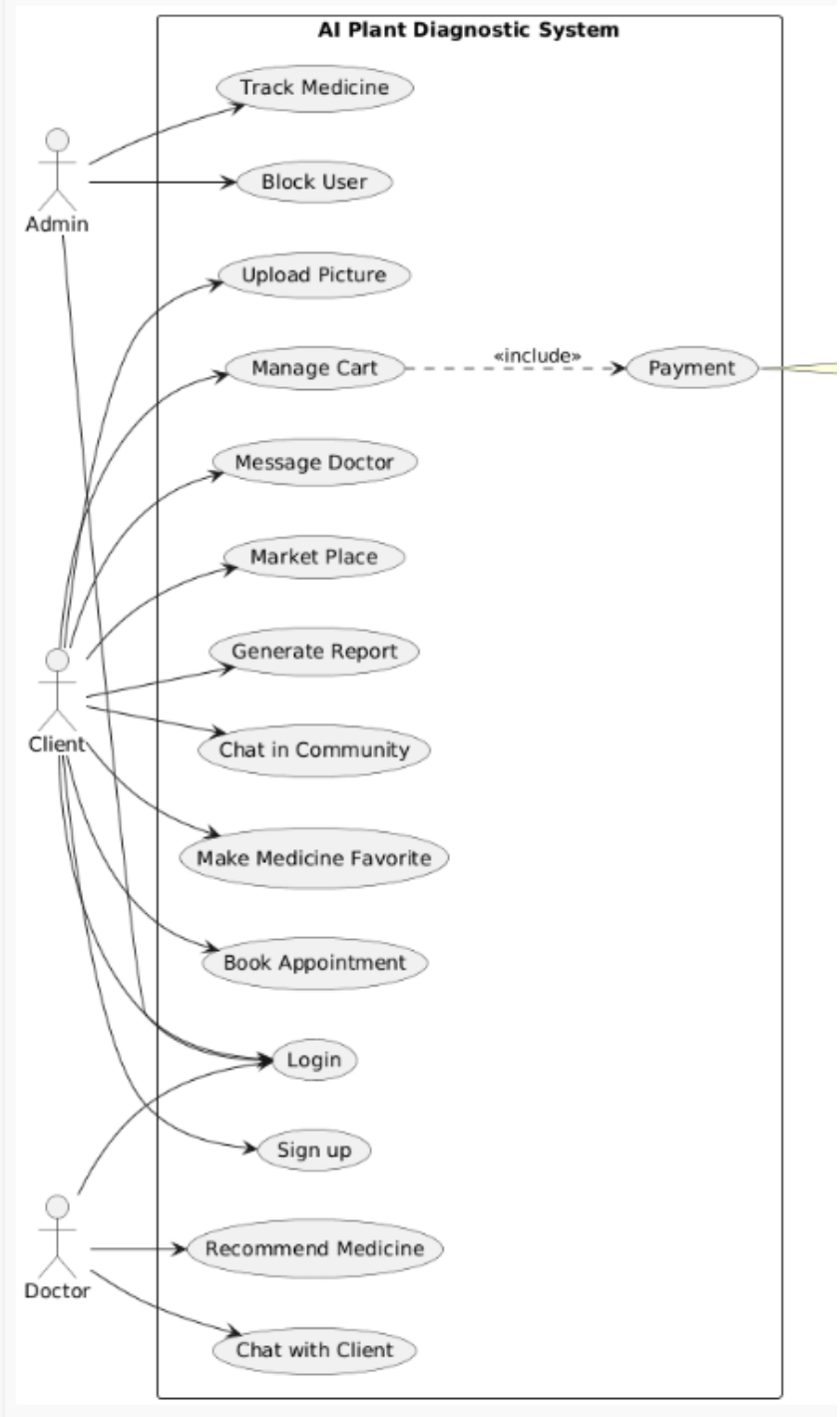

Figure 3:Use case diagram

3.6 Descriptive Use Case

3.6.1 Client Login

Table 2: Client Login

Name	Login		
Actors	Client		
Summary	The client logs into their account.		
Pre-Conditions	1. Client must have a registered account. 2. Client is on the Login screen.		
Post-Conditions	Client is successfully logged in and redirected to the Home Screen/Dashboard.		
Special Requirements	None		
Basic Flow			
Actor Action		**System Response**	
1	Client enters login credentials	2	System confirms credentials
3		4	Login successful.
Alternative Flow			
3	If details are incorrect, client must re-enter details.	4-A	The system responds with an error message

3.6.2 Client Upload Picture

Table 3: Client upload picture

Name	Upload Picture
Actors	Client
Summary	The client uploads a picture of their plant for diagnosis.
Pre-Conditions	Client has an account and is logged in.
Post-	Picture is uploaded and ready for analysis.

Conditions			
Special Requirements	None		
Basic Flow			
Actor Action		**System Response**	
1	Client logs in	1	Client logs in
3	Client uploads picture	3	Client uploads picture
Alternative Flow			
3	The user enters invalid email or password.	3	The user enters invalid email or password.

3.6.3 Client Sign Up

Table 4: Client Sign Up

Name	Sign Up		
Actors	Client		
Summary	The client creates a new account.		
Pre-Conditions	None.		
Post-Conditions	Client has a new account.		
Special Requirements	None		
Basic Flow			
Actor Action		**System Response**	
1	Client enters account details	2	System confirms credentials
		4	Account created
Alternative Flow			
3	If details are incorrect, client must re-enter details.	4-A	The system responds with an error message:.

3.6.4 Admin View User

Table 5: Admin view user

Name	View User		
Actors	Admin		
Summary	The admin views user details.		
Pre-Conditions	Admin is logged in.		
Post-Conditions	User details are displayed.		
Special Requirements	None		
Basic Flow			
Actor Action		**System Response**	
1	Admin selects user	2	System retrieves user details.
		4	Display details
Alternative Flow			
3	If user not found.	4-A	It notifies admin.

3.6.5 Generate Report

Table 6: Generate report

Name	Generate Report		
Actors	Client		
Summary	The Client generates and views a detailed report		
Pre-Conditions	client is logged in.		
Post-Conditions	Report is generated.		
Special Requirements	None		
Basic Flow			
Actor Action		**System Response**	

1	User selects report type.	2	Display report.
Alternative Flow			
3	If report generation fails	4-A	Error is generated.

3.6.6 Block User

Table 7: Block user

Name	Block User		
Actors	Admin		
Summary	The admin blocks a user from using the system.		
Pre-Conditions	Admin is logged in and user account exists.		
Post-Conditions	The user's account status is set to Blocked, preventing them from logging in		
Special Requirements	None		
Basic Flow			
Actor Action		**System Response**	
1	Admin selects user	2	System displays user and block button.
3	Admin confirms block action	4	System blocks user.
Alternative Flow			
3	If block fails, admin tries again or contacts support	4-A	Support contact details are given.

3.6.7 Manage Cart

Table 8: Mange cart

Name	Manage Cart
Actors	Client
Summary	The client adds, remove items from their shopping cart.
Pre-Conditions	Client is logged in.
Post-	Cart is updated with current items, and the client can proceed .

Conditions			
Special Requirements	None		
Basic Flow			
Actor Action		**System Response**	
1	Client adds items to cart	2	Display prompt that item is added.
3	Client remove items.	4	System updates cart
Alternative Flow			
3	If cart updating fails	4-A	Try again is prompted

3.6.8 Message Doctor

Table 9: Message Doctor

Name	Message Doctor		
Actors	Client		
Summary	Client sends a message to a doctor for advice.		
Pre-Conditions	Client booked an appointment.		
Post-Conditions	Message is sent.		
Special Requirements	None		
Basic Flow			
Actor Action		**System Response**	
1	Client composes message and sends it	2	System delivers the message to respective Doctor.
Alternative Flow			
3	If message sending fails	4-A	Generate an error

3.6.9 Message Client

Table 10: Message client

Name	Message Client

Actors	Doctor		
Summary	Doctor sends a message to client of advice.		
Pre-Conditions	Doctor conformed the appointment.		
Post-Conditions	Message is sent.		
Special Requirements	None		
Basic Flow			
Actor Action		**System Response**	
1	Doctor composes message and Sends it	2	System delivers the message to respective client.
3		4	
Alternative Flow			
3	If message sending fails	4-A	Generate an error

3.6.10 Chat in Community

Table 11: chat in community

Name	Chat in Community		
Actors	Client		
Summary	The client participates in community discussions.		
Pre-Conditions	Client is logged in.		
Post-Conditions	Chat session started.		
Special Requirements	None		
Basic Flow			
Actor Action		**System Response**	
1	Client enters community	2	System allows to enter community
3	Client participates in chat	4	System shows everyone the messages.
Alternative Flow			

3	If message sending fails	4-A	Generate an error

3.7 Sequence diagram

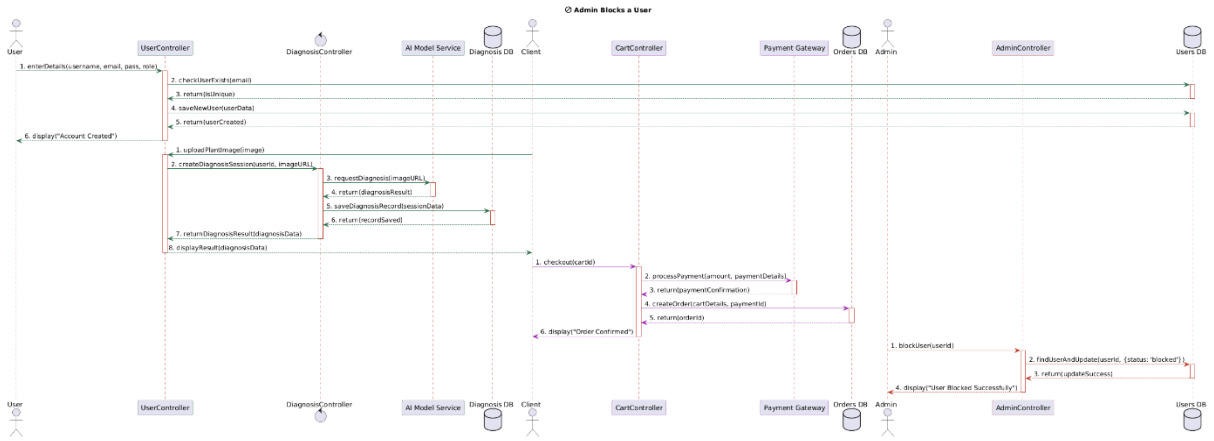

Figure 4: Sequence diagram

3.8 GUI Graphical User Interfaces

3.8.1 Login Screen

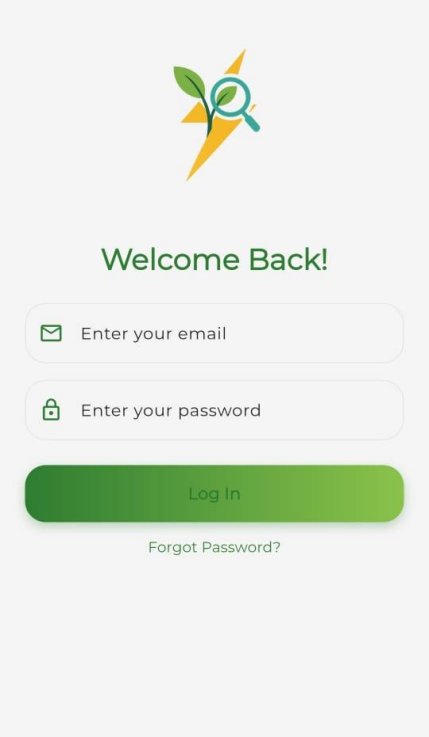

Figure 5: login screen

This is the application's Login Screen, featuring a clean, minimalist UI optimized for quick authentication. It displays the brand logo and the welcoming message, "Welcome Back!". The screen includes standard input fields for email and password with relevant icons, promoting clarity and ease of use. The main action is provided by a highly visible, gradient-green "Log In" button.

3.8.2 Sign Up Screen

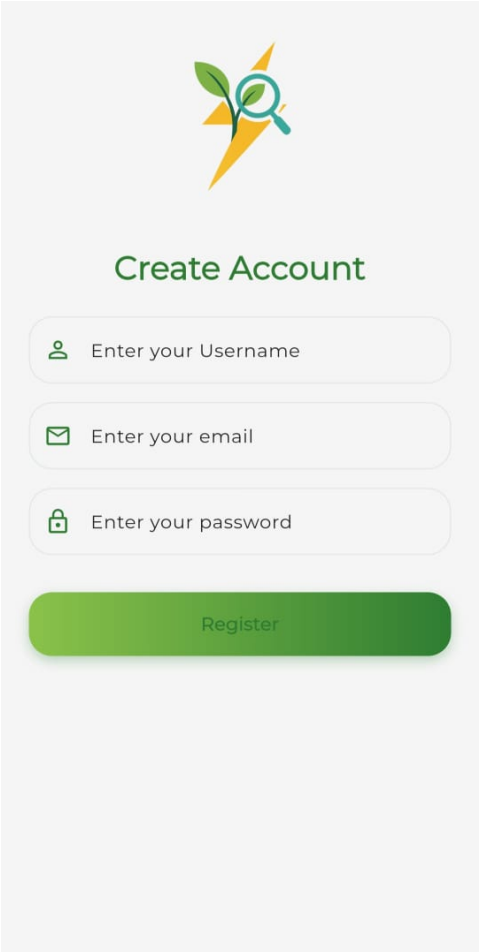

Figure 6: signup screen

This screen is the User Registration Module, maintaining the application's clean aesthetic and brand identity. It instructs the user to "Create Account" below the familiar app logo. The screen features three essential input fields: Username, Email, and Password, each marked by clear, distinct icons. The primary action is completed via the prominent, green "Register" button, facilitating new user enrollment. This design ensures a quick and straightforward account creation process.

3.8.3 Home Screen

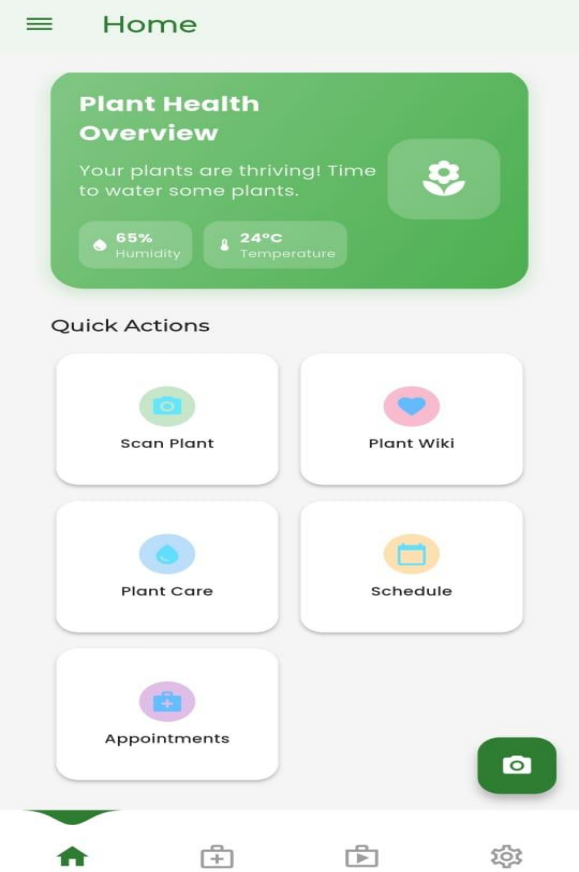

Figure 7: Home Screen

This image illustrates the application's Home Screen, designed as a central hub for all user interactions related to plant management. The screen adheres to modern mobile design principles, featuring a clean white background and organized information presented in distinct cards. At the top, the Plant Health Overview card uses a prominent green background to display summarized status information and relevant environmental metrics Humidity and Temperature. Below this, the Quick Actions section serves as the primary navigation area, presenting five essential features including "Scan Plant," "Plant Wiki," "Plant Care," "Schedule," and "Appointments" each accessible via a tap, distinctly colored icon card for maximized tap accuracy and visual separation.

3.8.4 Drawer

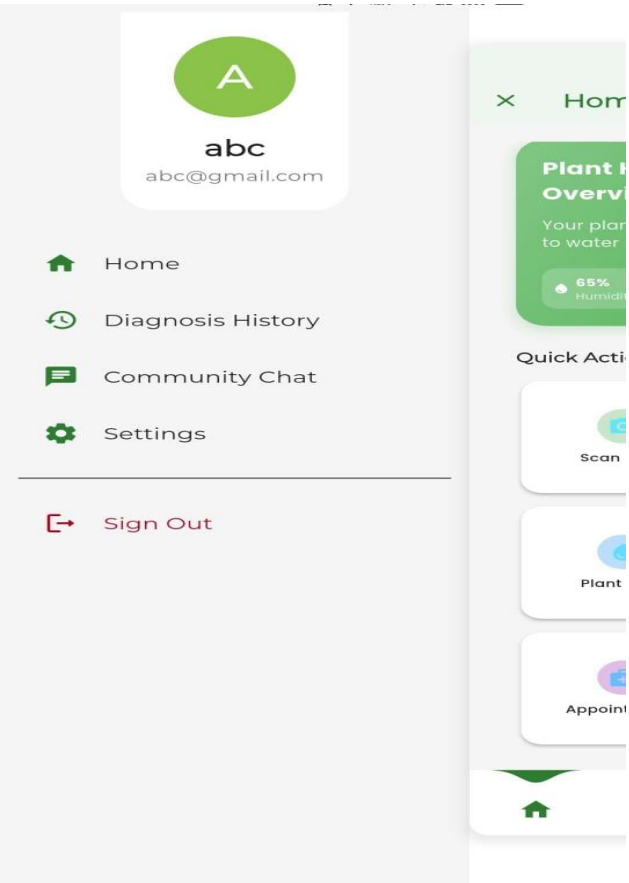

Figure 8: modern drawer screen

This image shows the Side Navigation Drawer, accessed via the hamburger menu icon on the Home Screen. The top section clearly features the User Profile with the user's name (and associated email address The main menu provides structured access to core application features: Home, Diagnosis History, Community Chat, and Settings. The menu is concluded by a visually separated and prominent Sign Out option, ensuring account security management.

3.8.5 Store Screen

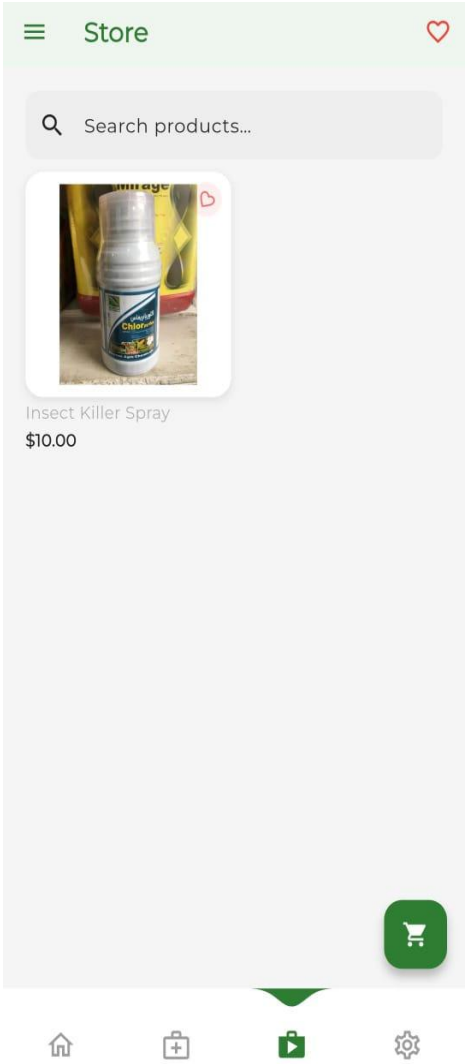

Figure 9: Store screen

This screen represents the application's E-commerce Store Interface, which is accessible via the main navigation. The top bar maintains the consistent design with a menu icon on the left and a Heart icon on the right, which functions as the gateway to the user's favorite or wish-listed products. A prominent, full-width Search Bar allows users to efficiently locate products using the prompt. The main content area displays products using a card-based layout, with a visible example showcasing an item for purchase. Key e-commerce features are integrated into the product card, including a small Heart icon for saving items. Finally, the user experience is enhanced by a large, floating green Shopping Cart icon in the bottom-right corner, providing quick access to the order summary, and the screen is anchored by the persistent bottom navigation bar.

3.8.6 Favorites Screen

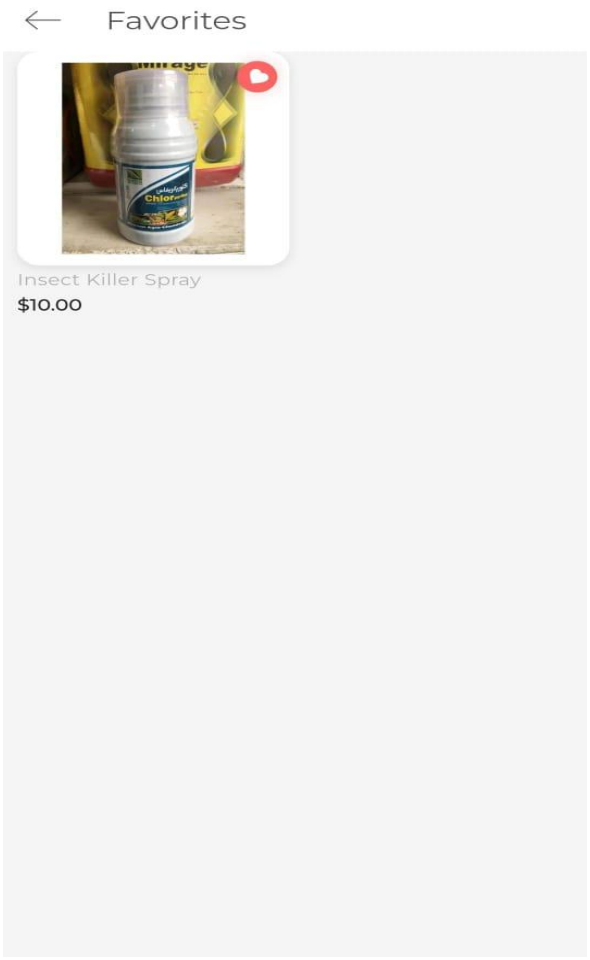

Figure 10: favorites screen

This screen represents the dedicated Favorites which is accessed directly from the store's main navigation. The top header clearly labels the screen "Favorites" and includes a simple Back Arrow for seamless navigation back to the previous screen (Store). The content area is structured to display user-selected products in a clear, accessible list format

3.8.7 Cart Screen

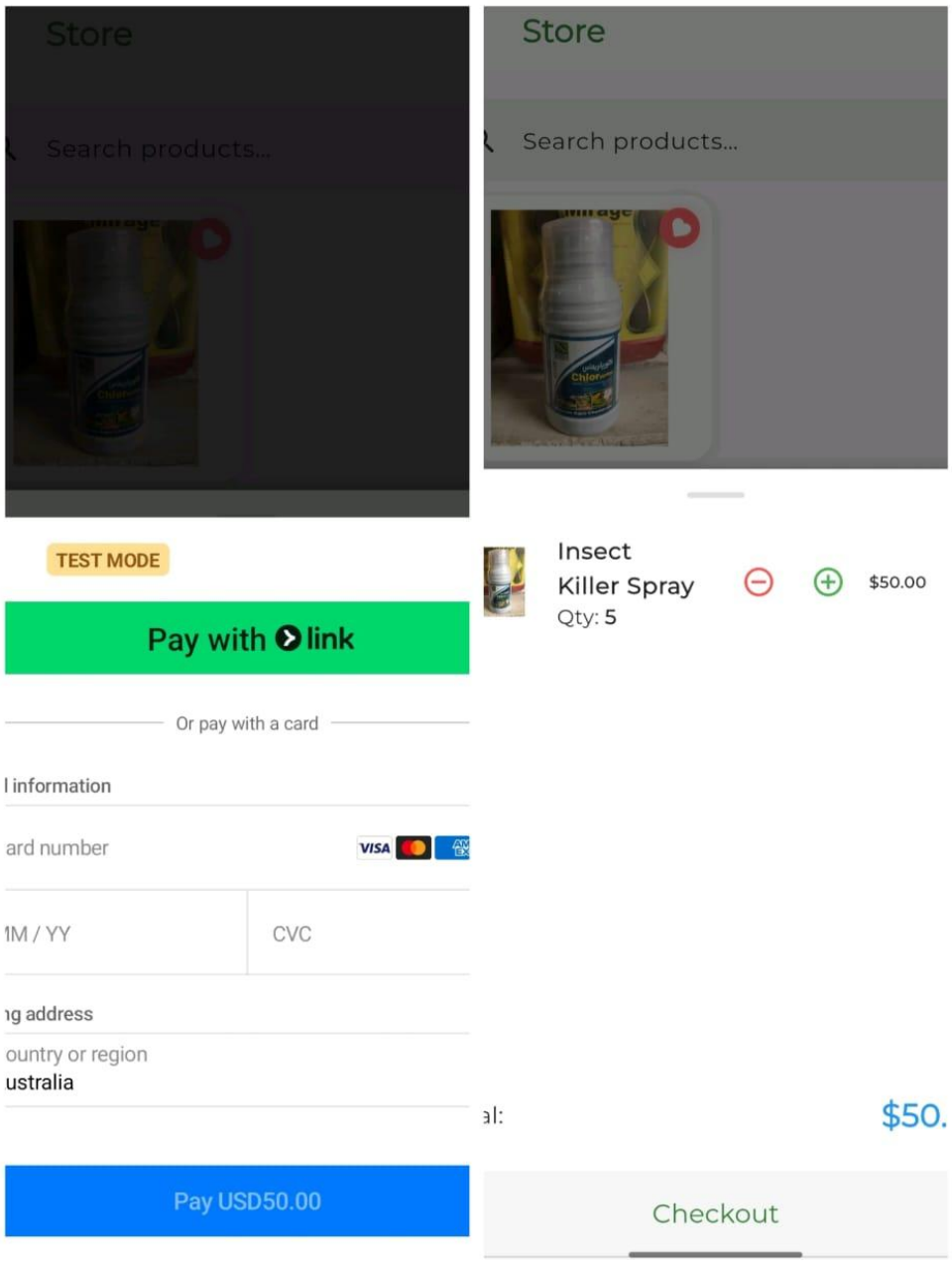

Figure 11: cart and check out screen

This image illustrates the Checkout Flow of the e-commerce module, combining the view of the cart summary and the payment interface. The right panel displays the Cart Summary, confirming the selected product) and its quantity with an associated total price .This view

uses clear + and - icons for quantity modification. Overlaid on the screen is the Payment Interface, presented as a bottom sheet. Below this is the option to pay with a card, prompting the user for standard credit card information number, expiry date (MM/YY), CVC), and billing address.

3.8.8 Doctor Screen

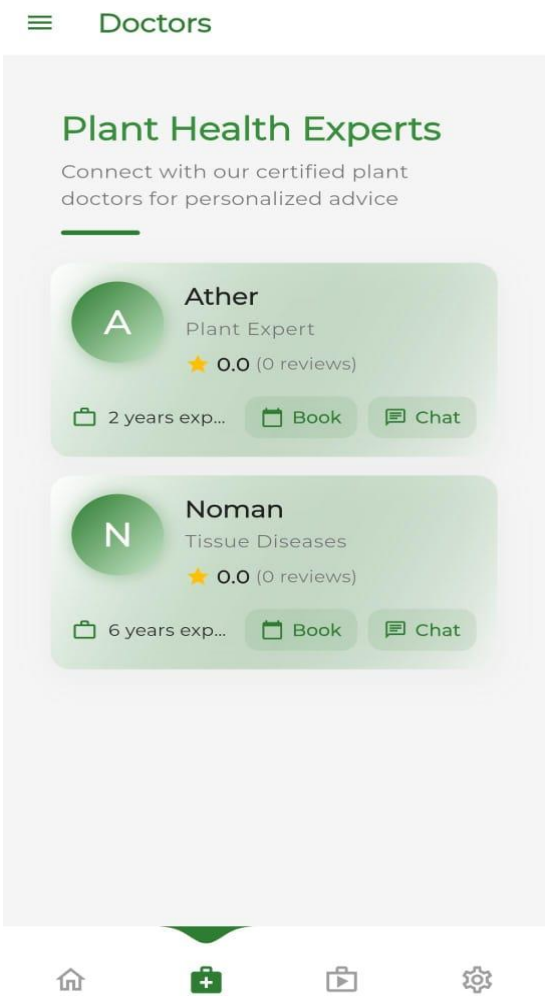

Figure 12: Doctor Screen

This screen is the Expert Consultation Module designed to connect users with qualified professionals for personalized advice.. Each profile card includes the expert's name, specialization, a rating and review summary, and key credentials (e.g., experience level). Crucially, each expert card features a clear the "Book" button inside with a calendar icon which directly facilitates the scheduling and confirmation of user appointments with the selected plant health expert.

3.8.9 Setting Screen

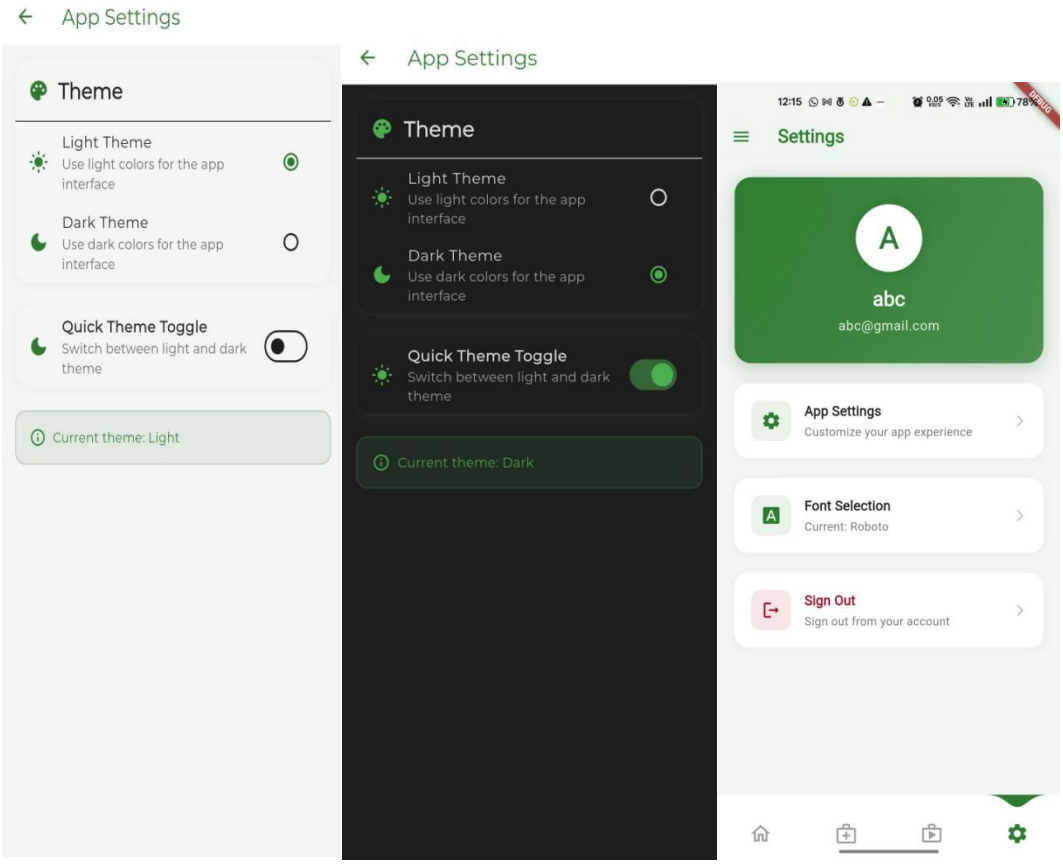

Figure 13: setting Screen

This image showcases the User Settings Module and the Theme Selection functionality. The main Settings screen begins with the user's profile information (avatar, email) and provides navigation to App Settings, Font Selection, and Sign Out. The Theme sub-screen allows the user to easily configure the application's visual appearance. Users can select between Light Theme or Dark Theme via radio buttons, supported by a quick toggle switch for rapid mode changes. This module provides necessary personalization and account management features.

3.8.10 Plant wiki screen

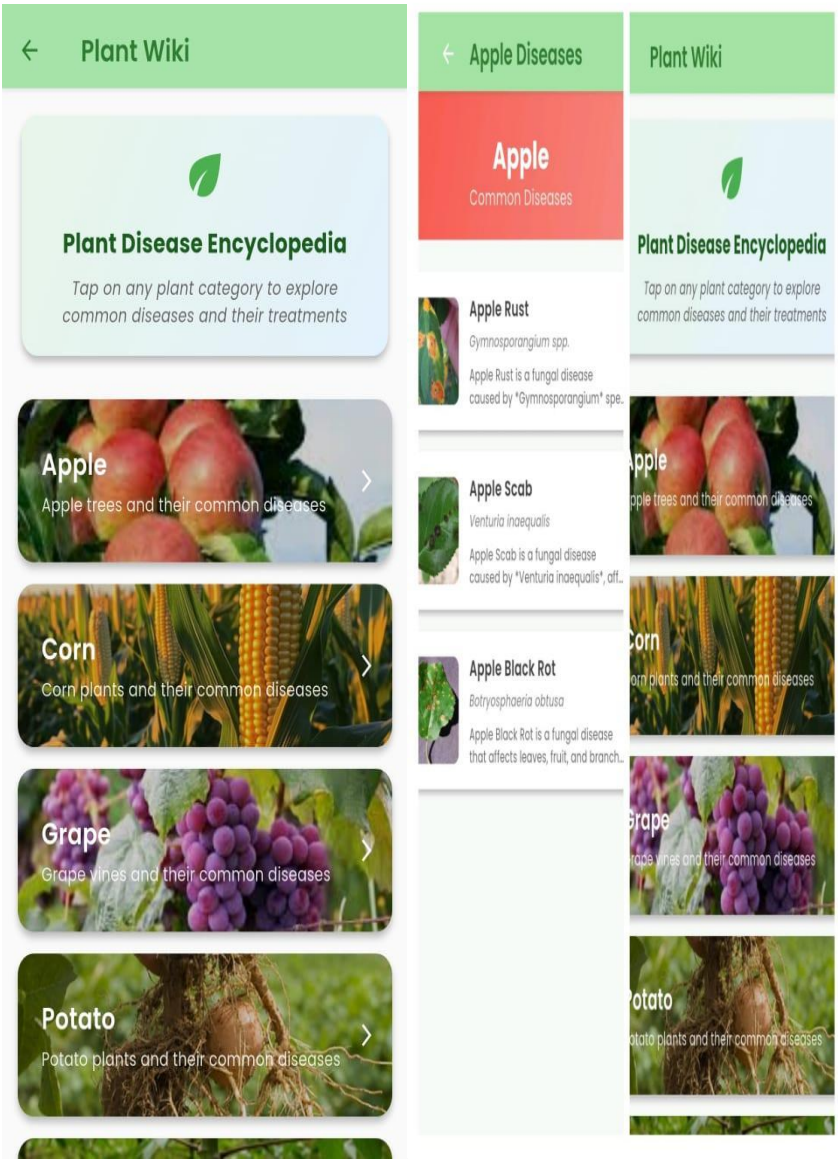

Figure 14: plant wiki screen

This composite image shows the Plant Disease Information Flow, starting with the Plant Wiki list. The list view categorizes diseases by plant and presents specific disease entries with brief descriptions for easy identification. Selecting an entry leads to the detailed Disease Detail Screen, which provides images, a description covering the cause and symptoms, and technical details and Care Tips, offering users direct, practical treatment advice for managing the plant disease.

3.8.11 Picture Upload Screen

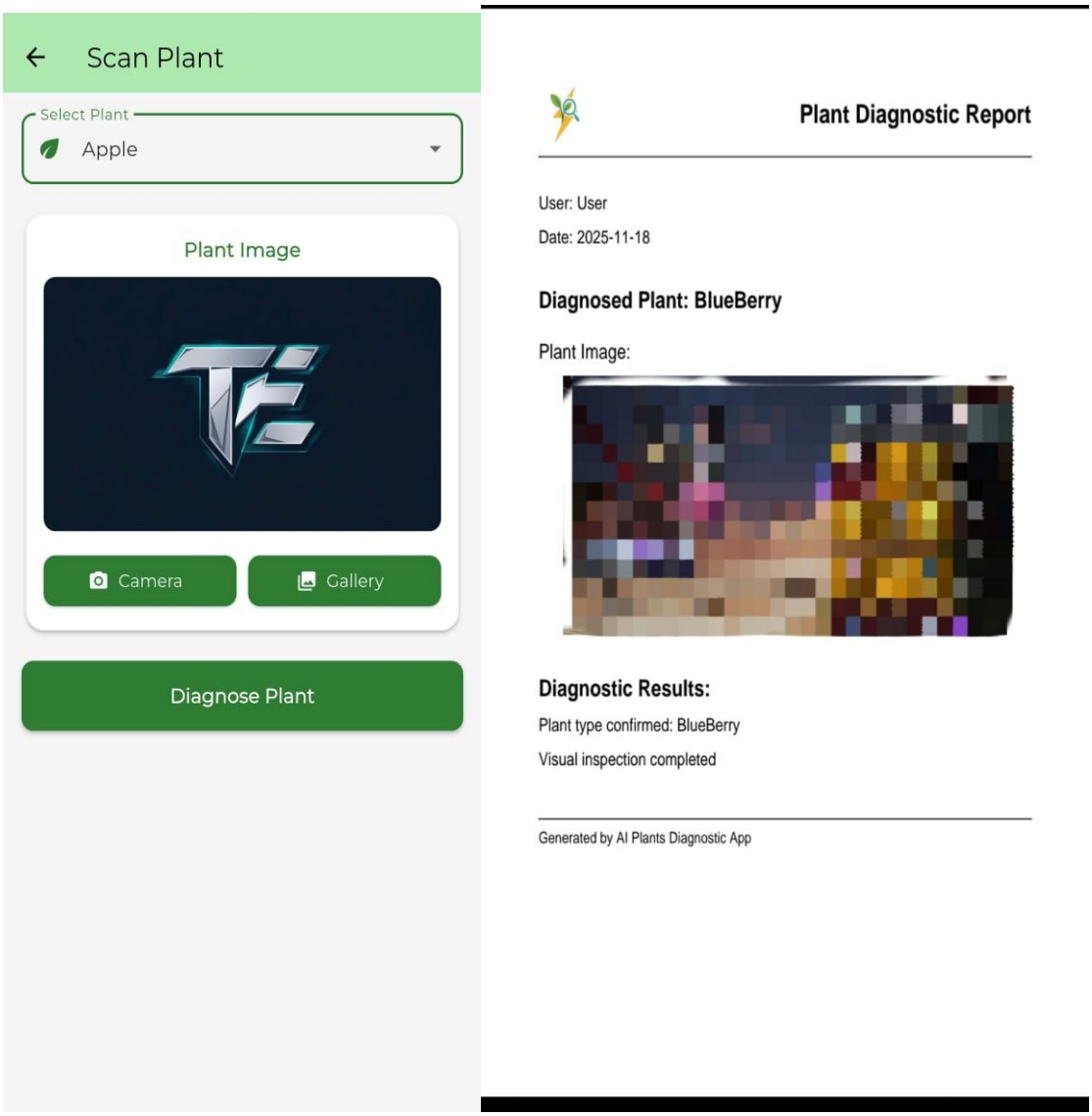

Figure 15: picture screen

This image illustrates the Plant Diagnosis Flow, beginning with the Scanning Input Screen where users select the plant type. Users submit a Plant Image using either the built-in camera or their photo gallery. The main action is the "Diagnose Plant" button, which initiates the automated analysis process. The results are displayed on the Diagnosis Report Screen, confirming the diagnosed plant type (e.g., Blueberry) and the completion of the visual inspection. This process enables users to quickly obtain AI-driven health assessments for their plants.

3.8.12 Community chat

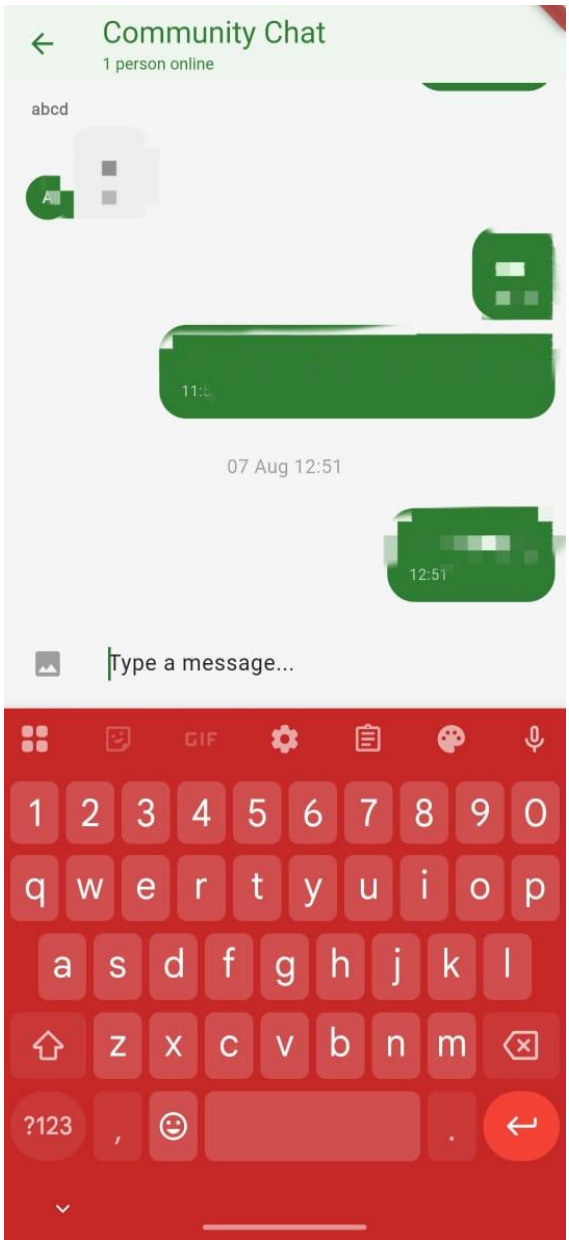

Figure 16: community chat

This screen displays the Community Chat interface, designed to facilitate user-to-user interaction and networking. The header includes the screen title, a back arrow, and the current online status of participants. The conversation thread shows messages using distinct green bubbles for outgoing messages and lighter bubbles for incoming messages. The bottom area features a text input field, an image attachment icon, and an active keyboard for instant communication.

3.8.13 Doctor appointment screen

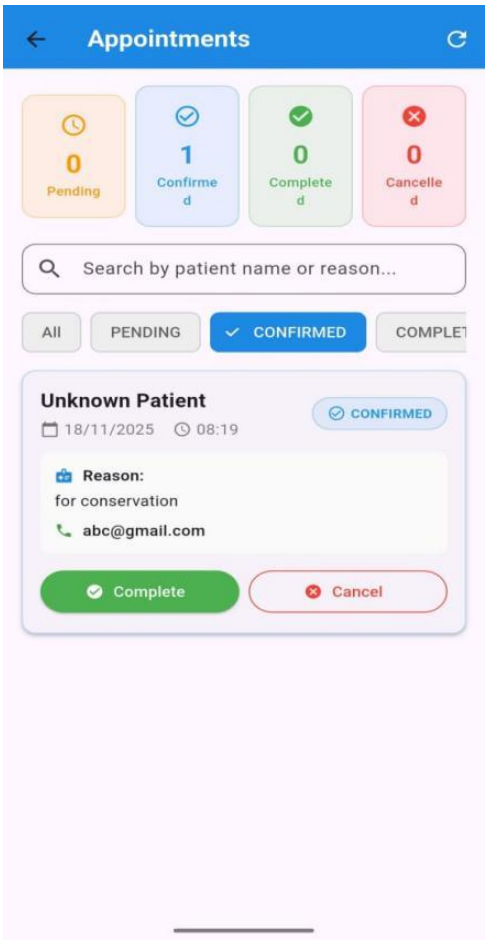

Figure 17: Doctor appointment screen

This screen serves as the Appointments Management Dashboard for the expert or "doctor." It features a clear Status Overview at the top, summarizing Pending, Confirmed, Completed, and Cancelled appointments with corresponding colors. A search bar and filter tabs (e.g., Confirmed) allow for efficient organization and lookup of consultations. Individual appointment cards display details (date, time, reason, email) and provide Complete and Cancel action buttons for managing the consultation status.

3.8.14 Doctor Chat Screen

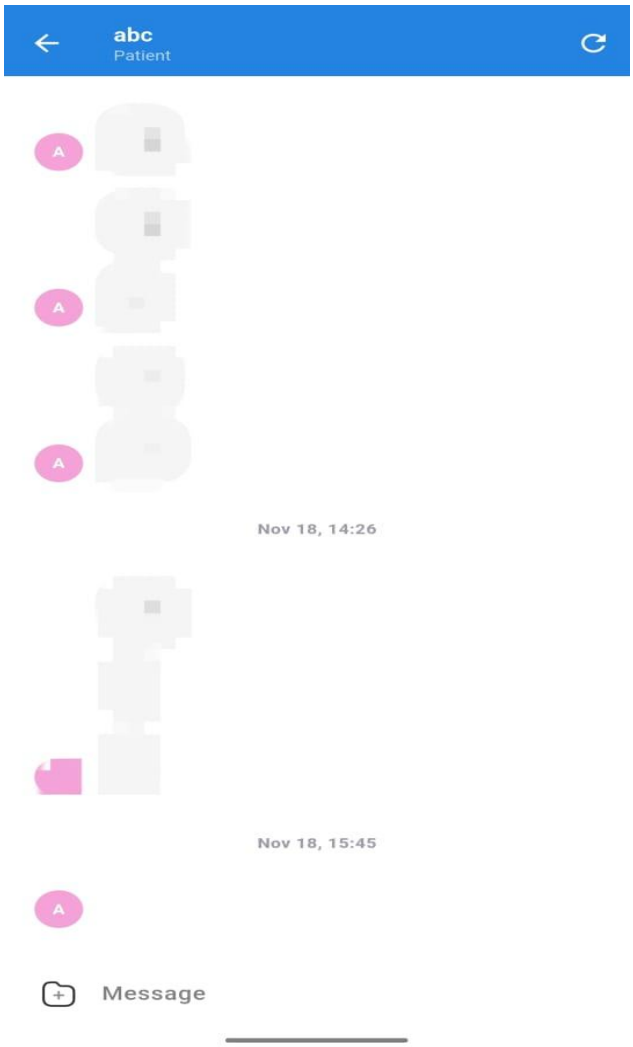

Figure 18: doctor chat screen

This screen presents a Direct Messaging interface for one-on-one communication (e.g., Doctor-Patient). The header identifies the recipient as "abc (Patient)" and features navigation controls (back arrow, refresh icon). The main body shows the conversation thread with messages time-stamped, using an avatar to identify the sender/recipient. At the bottom, a clear "Message" input field and icon for media attachments allow for easy message composition.

Chapter 4: Implementation and Test Cases

4.1 Introduction

The implementation of the proposed system for plant leaf disease classification involves the development of a hybrid CNN model with feature-level fusion aimed at accurately identifying and classifying various plant diseases. The model is designed to process leaf images through multiple stages, starting with the pre-processing of input images, followed by the feature extraction process using advanced Convolutional Neural Networks (CNNs). The system classifies diseases based on visual patterns and anomalies in the leaf images, with each disease category representing a distinct class. This section details the step-by-step methodology, including dataset preparation, model architecture, model training, and evaluation. The goal of this system is to provide a reliable and automated solution for identifying plant diseases, which can significantly aid in crop management and disease prevention.

4.2 Implementation

4.2.1 Proposed Framework

The proposed framework for plant leaf disease classification employs a hybrid CNN model with feature-level fusion, integrating the strengths of two state-of-the-art Convolutional Neural Networks (CNNs): ResNet-50 and InceptionV3. In this framework, an input image of size 224x224x3 is fed into both networks simultaneously. Each model processes the image, extracting distinctive features through their respective convolutional layers. These features are then aggregated using Global Average Pooling to reduce the output to a 2048-dimensional vector for each model. The key innovation in this architecture is the feature-level fusion, where the 2048-dimensional feature vectors from both networks are flattened and concatenated, creating a unified 4096-dimensional feature vector. This approach allows the model to combine the complementary information from both ResNet-50 and InceptionV3, enhancing its ability to learn from diverse visual patterns in plant leaf images.

Once the feature vectors are concatenated, the combined 4096-dimensional vector is passed through a linear classifier. This classifier maps the high-level features to the final disease categories, reducing the dimensionality from 4096 to the number of output classes. The model then applies a Softmax layer to convert the raw scores into probabilities, with the highest probability indicating the predicted disease class for the input image. By leveraging feature-level fusion, the hybrid CNN model combines the benefits of both ResNet-50 and

InceptionV3, offering improved classification accuracy and robustness. This architecture allows the system to effectively handle the complexity of plant leaf disease identification, providing a reliable solution for real-time agricultural monitoring and intervention.

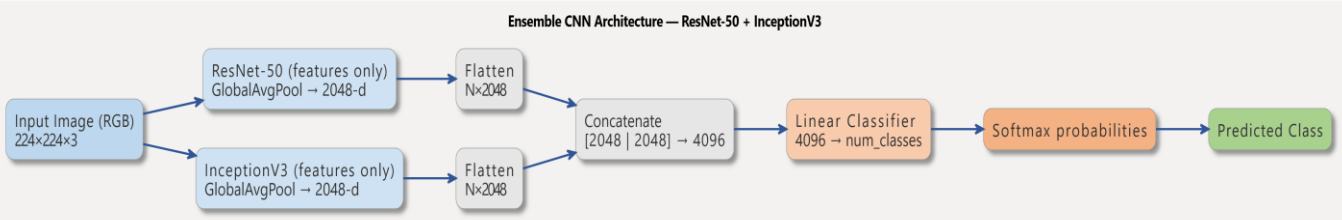

Figure 19: Proposed Architecture

4.2.2 Dataset Acquisition

For the plant leaf disease classification task, we utilized publicly available datasets, specifically the PlantVillage and Cassava Leaf Disease (CLD) benchmark datasets, summarized in table 11, which provide a rich set of images for model training and evaluation. These datasets cover a wide range of crops and diseases, offering good quality images for various plant species as shown in figure 20. By employing these datasets, we ensure that the model is exposed to a diverse array of plant diseases, facilitating robust training across 35multiple classes and improving the model's ability to generalize to unseen data.

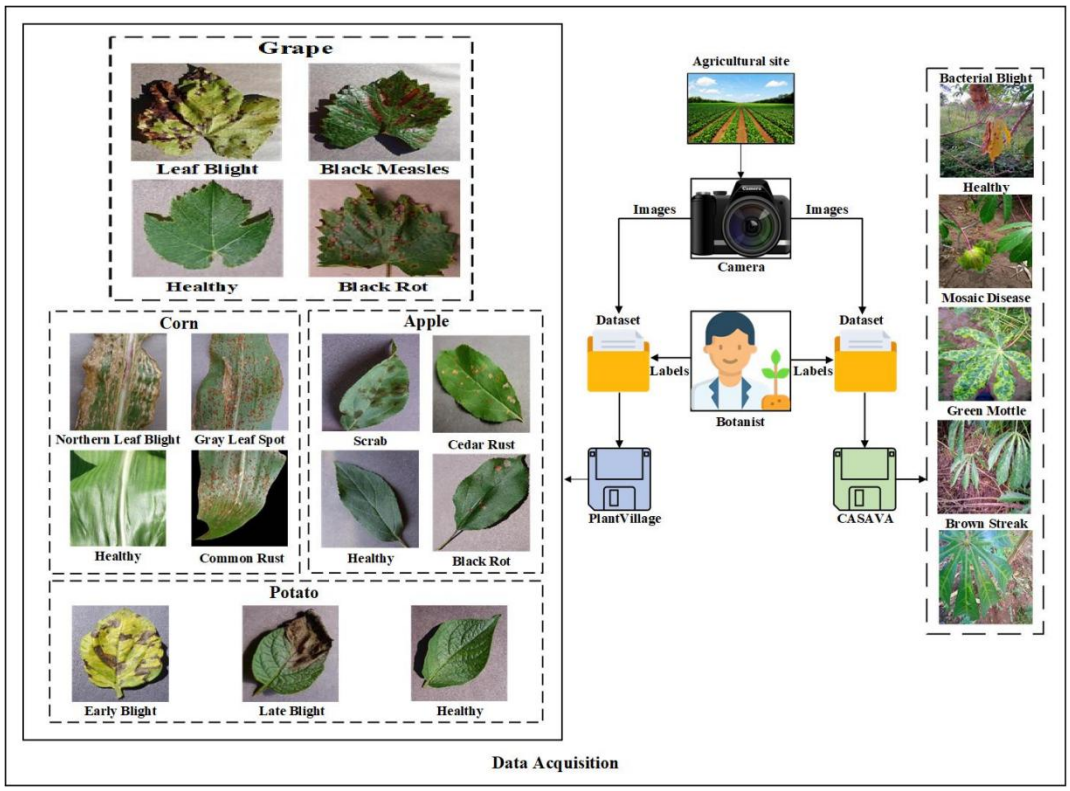

Figure 20: Data Acquisition

4.2.2.1 Plant Village

The PlantVillage dataset is a widely recognized resource for plant disease classification, from which we utilized aprroximately 13,908 images from various plant species. This dataset covers a broad spectrum of diseases, offering annotated images across multiple classes, which is essential for training and evaluating a multi-class classification model. The dataset includes images from plants such as apples, corn, grapes, potatoes. For example, the apple class is divided into categories like Apple Rust, Black Rot, Apple Scab, and Healthy; corn is divided into Northern Leaf Blight, Gray Leaf Spot, Common Rust, and Healthy; and grapes include Black Rot, Black Measles, Leaf Blight, and Healthy categories. The diversity of plants and diseases in the PlantVillage dataset provides a comprehensive basis for the study, ensuring that the proposed model can handle a variety of plant species and their associated diseases, as seen in Figure 20 .

4.2.2.2 Casava Leaf Disease

The Cassava Leaf Disease (CLD) dataset focuses on cassava, a vital staple crop in many developing countries. Cassava leaves are vulnerable to several diseases, which can have a severe impact on crop yield and food security. For this research, we selected 500 images per class from the CLD dataset, totaling 2,500 images across five distinct classes: Healthy, Cassava Brown Streak Disease (CBSD), Cassava Mosaic Disease (CMD), Cassava Green Mottle (CGM), and Cassava Bacterial Blight (CBB). These images, which include both healthy and diseased cassava leaves, are crucial for evaluating the model's ability to classify disease patterns in one of the world's most important crops. The CLD dataset ensures the model's applicability in real-world agricultural settings, specifically for cassava, highlighting its relevance for disease detection in developing countries where cassava is a major food source, as shown in Figure 20.

Table 12: Dataset Description

Plants	Classes	Image Frequency			
		Train (70%)	Validate (10%)	Test (20%)	Total
Corn	Northern Leaf Blight	689	98	198	985
	Gray Leaf Spot	359	51	103	513
	Healthy	813	116	233	1162
	Common Rust	834	119	239	1192
Apple	Apple Scab	441	63	126	630
	Healthy	1151	164	330	1645

	Apple Rust	192	27	56	275
	Black Rot	434	62	125	621
Grape	Leaf blight	753	107	216	1076
	Black Measles	968	138	277	1387
	Healthy	296	42	85	423
	Black Rot	826	118	236	1180
Potato	Early Blight	700	100	200	1000
	Late Blight	700	100	200	1000
	Healthy	106	15	31	152
Cassava	Cassava Green Mottle	350	50	100	500
	Cassava Bacterial Blight	350	50	100	500
	Cassava Brown Streak	350	50	100	500
	Cassava Mosaic Disease	350	50	100	500
	Healthy	350	50	100	500

4.2.2.3 Preprocessing

Figure 20 illustrates the process of standardizing image sizes, which is crucial for ensuring consistency across the dataset, reducing computational load, and enhancing the model's performance by providing uniform input images. In this study, we employ bicubic interpolation, a method that computes new pixel values by using the weighted sum of 16 neighboring pixels. This technique offers smoother transitions and superior image quality compared to other interpolation methods, such as nearest-neighbor or bilinear interpolation. Bicubic interpolation is particularly effective at preserving smooth gradients, which helps retain finer details and improves the accuracy of feature extraction during model training, ultimately contributing to better disease classification performance.

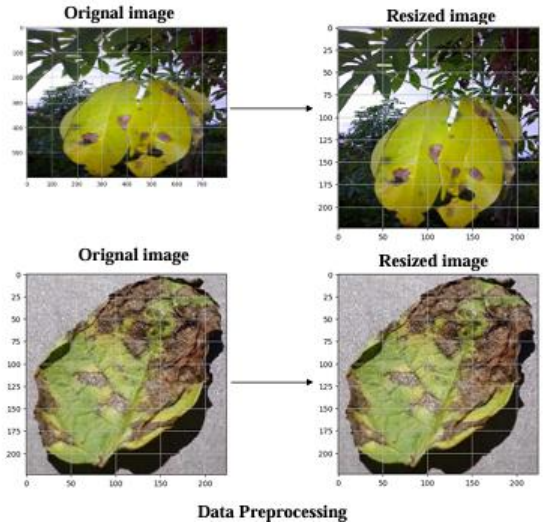

Figure 21: Image Resizing

4.2.3 Loss Function

To effectively train our proposed model for the plant disease classification, we used the sparse categorical cross entropy loss function. The formula for the loss function is defined in the equation 1.

$$Loss = -\sum_{i=1}^{n} t_i \log(p_{i,y_i}) \qquad 1$$

4.2.4 Model Training

The training process was conducted on Google Colab, utilizing its GPU capabilities to accelerate computation. The plant leaf disease classification model was trained for 40 epochs with approximately 29 million trainable parameters. To optimize the model, the Adam optimizer was used due to its adaptive learning rate and efficient handling of sparse gradients. A batch size of 32 was selected to balance memory usage and training stability, while a learning rate of 0.001 was chosen to support gradual and stable convergence. This training configuration enabled the model to achieve effective learning and strong generalization for accurate leaf disease classification.

4.2.5 Model Evaluation

Evaluating the performance of the CNN model is essential to ensure its reliability and effectiveness. The evaluation process for our model will involve:

- **Testing on Unseen Images:** Assessing the model's ability to generalize by testing it on a separate set of images that were not included in the training phase.
- **Performance Metrics:** Calculating key classification metrics such as accuracy, precision, recall, and F1 score to provide a comprehensive evaluation of the model's performance.

$$\boldsymbol{Precision} = \frac{TP}{TP+FP} \quad \textbf{2}$$

$$\boldsymbol{Recall} = \frac{TP}{TP+FN} \quad \textbf{3}$$

$$\boldsymbol{F1-score} = 2 \times \frac{(Recall \times Precision)}{(Recall+Precision)} \quad \textbf{4}$$

$$\boldsymbol{Accuracy} = \frac{TP+TN}{TP+TN+FN+FP} \quad \textbf{5}$$

4.3 Test case Design and description

4.3.1 Test case No.1

Table 13: client signup test case

AI Plant Diagnoses /client/ Sign up Module			
FYP II Documentation Section 4.2.1			
Test Case ID:	*TA-01*	**Test Date:**	*10/31/2025*
Test case Version:	*V1.0*	**Use Case Reference(s):**	*UC-Client Signup*
Revision History:	*NILL*		
Objective	*Testing the Signup module.*		
Product/Ver/Module:	*AI Plant Diagnoses/Client/Signup*		
Environment:	*Internet connectivity smart phone*		
Assumptions:	*Signup for the first time on system*		
Pre-Requisite:	*Have access the AI Plant Diagnoses Signup screen.*		

Step No.	**Execution description**	**Procedure result**
01	*Enter detail in the relevant fields with correct formatting. (Name: {A…. Z, a…. z}. Email: Contains '@' along with {{a, b, c…. z}, {A, B, C,….Z}, {1,2,3,….}} before '@' {.,_,-*	*System registers the user with pop-up indicating Signup successfully*

	} before '@' Ending with '.'And after those some alphabetic letters Password length >= 6 Contact: {1,2,3….}).	
02	*Enter invalid format of field "Name". {1,2,3….} {#, @ and other special characters} OR "Contact" Multiple or Special characters. OR the field "Password" Length < 6. OR the field "Email" Not contains '@' Multiple '@' characters Not ending at '.' Without letters*	*System indicates error message respectively "incorrect name format" , "incorrect contact", "incorrect password", "incorrect email". And doesn't register the use*
03	*Enter duplicate email which already registered in system.*	System indicate error "email already exist"
Comments: **Only Valid Formatting of the fields are accepted. Duplicate emails not accepted.**		
☐ *Passed* ☐ *Failed* ☐ *Not Executed*		

4.3.2 Test case No.2

.

Table 14: test case community chat

AI Plant Diagnoses/User/Community Chat			
FYP II Documentation Section 4.2.1			
Test Case ID:	*TA-02*	**Test Date:**	*10/31/2025*
Test case Version:	*V1.0*	**Use Case Reference(s):**	*UC Chat in Community*
Revision History:	*NILL*		
Objective	*Testing the Community Chat module.*		
Product/Ver/Module:	*AI Plant Diagnoses/User/Community Chat*		
Environment:	*Internet connectivity /*		
Assumptions:	*The user is already logged in and the community chat server is active.*		

Pre-Requisite:	*he user has successfully navigated to the Community Chat screen.*	
Step No.	**Execution description**	**Procedure result**
01	*Tap the Image/Attachment icon next to the input field, select a valid image file (e.g., JPEG, PNG) from the gallery, and send it*	*The image attachment is successfully processed and displayed within the chat window; a confirmation of upload success is visible.*
02	*Attempt to send an empty message (tap the Send button without typing any text).*	*The system prevents the message from being sent.*
03	*Observe the online status display at the top of the screen (e.g., "1 person online").*	*The online count is accurate and dynamically updates if another user logs into or out of the chat.*
Comments: The core functionality of message sending (text and media) must be validated, along with basic input sanitation (empty messages). Timestamps and bubble colors must be accurate.		
☐ *Passed* ☐ *Failed* ☐ *Not Executed*		

4.3.3 Test case No.3

Table 15: client doctor chat

AI Plant Diagnoses/User/Doctor Chat			
FYP II Documentation Section 4.2.1			
Test Case ID:	*TA-03*	**Test Date:**	*10/31/2025*
Test case Version:	*V1.0*	**Use Case Reference(s):**	*UC Client* Doctor Chat
Revision History:	*NILL*		
Objective	*Testing the functionality for users to send and receive messages, including media, in a one-on-one chat with a Plant Health Expert/Doctor.*		
Product/Ver/Module:	*AI Plant Diagnoses/User/Doctor Chat*		
Environment:	*Internet connectivity*		

Assumptions:	*The user and a Doctor are both logged in, and the user has initiated the chat or has a confirmed appointment.*	
Pre-Requisite:	*The user has successfully navigated to the Direct Chat screen with a Doctor and doctor has confirmed the appointment*	
Step No.	**Execution description**	**Procedure result**
01	*Tap the Attachment icon and select an image (e.g., a photo of an ailing plant) from the gallery to send.*	*The image attachment is successfully uploaded, sent, and displayed correctly within the chat thread, visible to the Doctor.*
02	*(Doctor's side action - check system behavior): Have the Doctor send a reply message to the user.*	*The Doctor's reply appears instantly in the user's chat window, displayed in the* ***incoming message bubble format*** *(different from the user's outgoing bubble) with their avatar and correct timestamp.*
Comments: Focus on secure, private, and real-time message exchange. Validation of attachment handling (especially images for diagnosis) and accurate message identification (via avatars) is critical.		
☐ *Passed* ☐ *Failed* ☐ *Not Executed*		

4.3.4 Test case No 4

.

Table 16: Admin Login

AI Plant Diagnoses / Admin/Login Module			
FYP II Documentation Section 4.2.1			
Test Case ID:	*TA-04*	**Test Date:**	*10/31/2025*
Test case Version:	*V1.0*	**Use Case Reference(s):**	*UC-Admin login*
Revision History:	*NILL*		
Objective	*Testing the Login module.*		
Product/Ver/Module:	*AI Plant Diagnoses/Admin/login*		
Environment:	*Internet connectivity / web*		
Assumptions:	*Admin data exists in the system database.*		

Pre-Requisite:	*Have access the AI Plant Diagnoses Admin login screen.*	
Step No.	**Execution description**	**Procedure result**
01	*Enter correct Email and password.*	*System matches the credentials from the database and indicates the pop-up message that "login successfully"*
02	*Enter invalid email and correct password.*	*System matches the credentials from database indicates error "incorrect email"*
03	*Enter valid email and incorrect password.*	*System indicates error "incorrect password"*
04	*Enter invalid email and invalid password.*	*System indicates error "incorrect email or password".*
05	*Enter credentials not registered in the system. OR enter name and password field remains empty.*	*System indicates error "user doesn't exist"*
Comments: **Only valid Admin can login to the system who registered himself prior.**		
☐ *Passed* ☐ *Failed* ☐ *Not Executed*		

4.3.5 Test case No 5

Table 17: client login test case

AI Plant Diagnoses / Client/Login Module			
FYP II Documentation Section 4.2.1			
Test Case ID:	*TA-05*	**Test Date:**	*10/31/2025*
Test case Version:	*V1.0*	**Use Case Reference(s):**	*UC-client login*
Revision History:	*NILL*		
Objective	*Testing the Login module.*		
Product/Ver/Module:	*AI Plant Diagnoses/Client//login*		
Environment:	*Internet connectivity /smart phone*		
Assumptions:	*User data exists in the system database.*		
Pre-Requisite:	*Have access the AI Plant Diagnoses login screen.*		
Step No.	**Execution description**	**Procedure result**	

01	*Enter correct Email and password.*	*System matches the credentials from the database and indicates the pop-up message that "login successfully"*
02	*Enter invalid email and correct password.*	*System matches the credentials from database indicates error "incorrect email"*
03	*Enter valid email and incorrect password.*	*System indicates error "incorrect password"*
04	*Enter invalid email and invalid password.*	*System indicates error "incorrect email or password".*
05	*Enter credentials not registered in the system. OR enter name and password field remains empty.*	*System indicates error "user doesn't exist"*
Comments: **Only valid user can login to the system who registered himself prior.**		
☐ *Passed* ☐ *Failed* ☐ *Not Executed*		

4.3.6 Test case No 6

Table 18: Doctor login test case

AI Plant Diagnoses / Doctor/Login Module			
FYP II Documentation Section 4.2.1			
Test Case ID:	*TA-06*	**Test Date:**	*10/31/2025*
Test case Version:	*V1.0*	**Use Case Reference(s):**	*UC-Doctor login*
Revision History:	*NILL*		
Objective	*Testing the Login module.*		
Product/Ver/Module:	*AI Plant Diagnoses/Doctor//login*		
Environment:	*Internet connectivity /smartphone*		
Assumptions:	*Doctor data exists in the system database.*		
Pre-Requisite:	*Have access the AI Plant Diagnoses Login login screen.*		
Step No.	**Execution description**	**Procedure result**	
01	*Enter correct name*	*System matches the credentials from the database and indicates the pop-up message that "login successfully"*	

02	*Enter invalid name*	*System matches the credentials from database indicates error "invalid login"*
Comments: **Only valid doctor can login to the system who registered himself prior.**		
☒ *Passed* ☐ *Failed* ☐ *Not Executed*		

4.3.7 Test case No 7

Table 19: Book Appointment Module test case

AI Plant Diagnoses / Client/Book Appointment Module			
FYP II Documentation Section 4.2.1			
Test Case ID:	*TA-07*	**Test Date:**	*10/31/2025*
Test case Version:	*V1.0*	**Use Case Reference(s):**	*UC-Appointment booking*
Revision History:	*NILL*		
Objective	*Testing the Online appointment booking module for the client.*		
Product/Ver/Module:	*AI Plant Diagnoses/Client//Book Appointment*		
Environment:	*Internet connectivity /web*		
Assumptions:	*User data exists in the system database.*		
Pre-Requisite:	*Must Login to the system and doctor schedule the appointment.*		
Step No.	**Execution description**	**Procedure result**	
01	*Patient select a doctor and then select specific date and time.*	*System sent the appointment request to the doctor. And indicate the message "appointment successfully booked"*	
02	*Patient try to select date or time which is already booked for that doctor.*	*System force him to select only available time and date.*	
Comments: **appointments will be booked**			
☒ *Passed* ☐ *Failed* ☐ *Not Executed*			

4.3.8 Test case No 8

Table 20: Client Report test case

AI Plant Diagnose/Client/Report			
FYP II Documentation Section 4.2.1			
Test Case ID:	*TA-07*	**Test Date:**	*10/31/2025*
Test case Version:	*V1.0*	**Use Case Reference(s):**	*UC-Client View*
Revision History:	*NILL*		
Objective	*Testing the report section for the client to view and download report.*		
Product/Ver/Module:	*AI Plant Diagnoses/Client/Report*		
Environment:	*Internet connectivity /smart phone*		
Assumptions:	*Client has uploaded the picture and selected the category.*		
Pre-Requisite:	*Patient must be registered and logged in*		

Step No.	**Execution description**	**Procedure result**
01	*Log in as a patient, navigate to the Prescription section, select a specific category with image, and click to download as PDF.*	*System generates and downloads the prescription as a PDF file.*

Comments:

Patients can only view and download report

☒ *Passed* ☐ *Failed* ☐ *Not Executed*

4.3.9 Test case No 9

Table 21: Admin/User Management test case

AI Plant Diagnose/Admin/User Management			
FYP II Documentation Section 4.2.1			
Test Case ID:	*TA-07*	**Test Date:**	*10/31/2025*
Test case Version:	*V1.0*	**Use Case Reference(s):**	*UC-Admin Block User*
Revision History:	*NILL*		
Objective	*Testing the admin's functionality to block a specific user account.*		
Product/Ver/Module:	*AI Plant Diagnoses/Admin/User Management*		
Environment:	*Internet connectivity /web*		

Assumptions:	*The user to be blocked exists in the system. The admin has the appropriate privileges.*	
Pre-Requisite:	*Admin must be registered and logged into the admin panel.*	
Step No.	**Execution description**	**Procedure result**
01	*Log in as an Admin and navigate to the "User Management" or "Clients" section.*	*Admin dashboard is displayed with a list of registered users.*
02	*Click the "Block" button or option associated with that user's account*	*The system blocks the user, and their status changes to "Blocked" in the user list. A success notification is displayed.*
Comments: **Once blocked, the user should not be able to log in or access any client-side functionalities.**		
☐ *Passed* ☐ *Failed* ☐ *Not Executed*		

4.3.10 Test case No 10

Table 22: Client/payment test case

AI Plant Diagnose/Client/payment			
FYP II Documentation Section 4.2.1			
Test Case ID:	*TA-07*	**Test Date:**	*10/31/2025*
Test case Version:	*V1.0*	**Use Case Reference(s):**	*UC-Client Payment*
Revision History:	*NILL*		
Objective	*To test the successful processing of a client's payment for a service using the integrated Stripe payment gateway.*		
Product/Ver/Module:	*AI Plant Diagnoses/Client/Payment*		
Environment:	*Internet connectivity /smart phone*		
Assumptions:	*The Stripe API is correctly integrated. The product has a defined price. The user has a valid payment method.*		
Pre-Requisite:	*The user must be registered and logged in. The user has selected a product that requires payment*		
Step No.	**Execution description**	**Procedure result**	

01	*Select the "Pay with Credit/Debit Card" option.*	*The system displays the Stripe payment form, prompting for card number, expiration date, and CVC.*
02	*Enter valid test payment credentials*	*The input fields accept the data without validation errors.*
03	*Click the "Pay Now" Payment button.*	*The system processes the transaction. Upon success, a "Payment Successful" confirmation is displayed,*
Comments: **A successful transaction should be logged in the admin's payment history, and an email receipt should be automatically sent to the client.**		
☒ *Passed* ☐ *Failed* ☐ *Not Executed*		

4.4 Summary

In this section, we summarize the common attributes and metrics used to evaluate the test cases conducted for the AI Plant Diagnose application, ensuring the reliability and functionality of its modules, such as user management, report generation, and payment processing. The test metrics provide a standardized framework to measure the success, efficiency, and coverage of the testing process. These metrics are derived from the attributes shared across all test cases and are essential for assessing the system's performance in real-world scenarios.

The following are the key test metrics and their common attributes:

1. **Test Case Objective:** Each test case is designed with a clear objective to validate specific functionalities of the system. For instance, the objective may include testing user management features, such as an admin's ability to block a user account (as in Test Case TA-08), or verifying that a client can successfully download a diagnosis report as a PDF (as in Test Case TA-07). This ensures that the test aligns with the functional requirements outlined in the use case references.
2. **Test Execution Status:** The outcome of each test case is categorized as "Passed," "Failed," or "Not Executed." This metric indicates whether the system behaves as expected under the defined conditions. For example, if Test Case TA-08 passed, it would confirm the correct functionality of the admin's user-blocking feature.
3. **Test Coverage**: This metric evaluates the extent to which the system’s functionalities are tested. Each test case targets a specific module (e.g., Admin User Management, Client Report, Client Payment) and verifies critical operations, ensuring comprehensive coverage of the system’s features. The use case references (e.g., UC-Admin Block User,

UC-Client View) link test cases to functional requirements, ensuring no critical feature is overlooked.

4. **Pre-requisites and Assumptions:** Common attributes include the pre-conditions required for executing the test, such as user authentication (e.g., an admin must be logged in, as in TA-08) and system state (e.g., the user account to be blocked must already exist in the system). These ensure the test environment is consistent and replicable.
5. **Procedure and Result Validation:** Each test case includes a step-by-step execution procedure and corresponding results. The results are validated against expected outcomes, such as a user's status changing to 'Blocked' after the admin's action or the successful download of a PDF file upon a client's request. This metric ensures that the system's behavior aligns with the intended design.
6. **Error Handling and Comments:** This attribute captures any issues, observations, or system limitations encountered during testing. Comments provide insights into the system's performance, such as the explicit requirement noted in Test Case TA-08: "Once blocked, the user should not be able to log in or access any client-side functionalities."
7. **Test Environment:** The test environment, including the platform (e.g., Web Browser, Smart Phone) and module (e.g., AI Plant Diagnose/Admin/User Management), is consistently documented to ensure tests are conducted under standardized conditions.This metric supports the reproducibility and scalability of the testing process.

These metrics collectively provide a quantitative and qualitative assessment of the system's functionality, enabling stakeholders to gauge the reliability and robustness of the AI Plant Diagnose application. By analyzing the outcomes of test cases, such as those for user management and report generation, we ensure that the system meets user requirements and performs effectively in managing critical operations.

4.5 Test Metrics

4.5.1 Test case Matric.No.1

Metric:	**Purpose**

Number of Test Cases:	10
Number of Test Cases Passed:	10
Number of Test Cases Failed:	0
Test Case Defect Density:	(0 x 100)/10=0
Test Case Effectiveness:	If all defects were found via test cases: (5/5 × 100) = 100%
Traceability Matrix:	All 10 test cases map directly to defined use cases. Traceability maintained

Chapter 5: Experimental Results and Analysis

5.1 Introduction

In this chapter, we present a comprehensive evaluation of our experimental results and analysis for the multi-class plant leaf disease classification . We will discuss the experiments conducted using benchmark Convolutional Neural Network (CNN) models, widely recognized for their effectiveness in image classification tasks. Additionally, we will highlight the performance of our proposed hybrid CNN model with feature-level fusion, designed to address the specific challenges of plant leaf disease classification. The results of our model will be compared against these benchmark CNNs to demonstrate its competitive performance . Through detailed analysis, we aim to provide insights into the strengths, and potential areas of improvement of our approach.

5.2 Experiments

To evaluate the effectiveness of the proposed hybrid CNN model for plant leaf disease classification, we conducted a series of structured experiments using the PlantVillage and Cassava Leaf Disease (CLD) datasets. The experiments were designed to assess the model's performance in terms of classification accuracy, robustness against variations in leaf images, and its ability to generalize across different plant species and disease types. The proposed model integrates ResNet-50 and InceptionV3 through feature-level fusion to enhance the extraction of multiscale features and reduce information loss, improving the overall performance in identifying plant diseases. To ensure a comprehensive evaluation, we compared the proposed model with several benchmark CNN architectures, including ResNet-50, ResNet-18, DenseNet201, InceptionV3, VGG16, and MobileNetV2 each known for their strong performance in image classification tasks. This section provides a detailed comparative analysis, including performance plots and confusion matrices, to assess the strengths and weaknesses of the proposed model.

5.2.1 Proposed model Performance Evaluation and Comparison on Multiclass Dataset

To thoroughly assess the performance of the proposed convolutional neural network (CNN) model designed for plant leaf disease classification, a series of analytical evaluations were

conducted. These include training and validation accuracy/loss analysis over epochs, confusion matrix interpretation, and class-wise performance based on precision, recall, and F1-score. The model was tested on the combined PlantVillage and CLD benchmark datasets, which includes twenty classes.From the PlantVillage dataset, four major crop species were selected: Corn, Apple, Grape, and Potato, each encompassing both healthy and diseased leaf images. For Corn, the classes include Common Rust, Gray Leaf Spot, Northern Leaf Blight, and Healthy leaves. Apple images consist of Apple Rust, Black Rot, Apple Scab, and Healthy. Grape classes cover Black Rot, Black Measles, Leaf Blight, and Healthy samples. Potato classes include Early Blight, Late Blight, and Healthy leaf images.

To complement this dataset and increase class diversity, images from the Cassava Leaf Disease Dataset were also included. Specifically, a balanced subset was formed by selecting 500 images per class from five cassava categories: Cassava Mosaic Disease, Cassava Green Mottle, Cassava Bacterial Blight, Cassava Brown Streak, and Healthy leaves. The comprehensive evaluation confirms the model's strong predictive capabilities and highlights both its strengths and areas where further enhancement could be beneficial.

The training and validation curves shown in the graphs in figure 22 provide valuable insights into the model's learning process over 40 epochs. In the Loss plot, we observe a significant fluctuation in both training and validation loss during the initial epochs, with validation loss peaking around epoch 10. However, after the early fluctuations, both training and validation losses decrease steadily. This suggests that the model initially struggled to generalize but gradually learned effective patterns. Despite the initial spikes, the validation loss stabilizes around epoch 30, remaining relatively low thereafter.

For the Accuracy plot, there is a notable increase in training accuracy, which rises sharply from the early epochs and continues to climb, reaching over 90% by the final epoch. The validation accuracy follows a similar trajectory, improving significantly in the first few epochs and stabilizing between 85–90% after around epoch 30. This indicates that the model learned quickly during the early epochs and continued to improve but struggled slightly to match the training performance, hinting at a potential gap in generalization.

Overall, the model's performance appears to improve steadily in terms of both accuracy and loss, with the final validation accuracy of 94.78% demonstrating effective learning despite some initial difficulties.

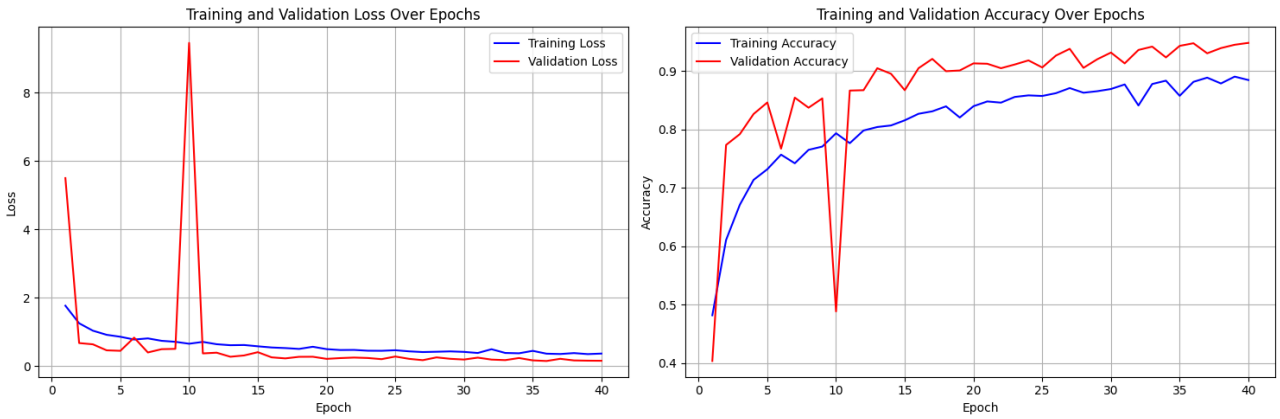

Figure 22: illustrating the plots of the proposed model

The confusion matrix presented here provides a detailed evaluation of the model's performance in classifying images of various plant diseases across multiple categories. The rows represent the true labels, while the columns represent the predicted labels. Each cell shows the number of instances where the true class was predicted as a particular class.

The confusion matrix shown in Figure 23 provides a detailed evaluation of the model's performance across each plant disease class. Apple_Healthy achieved the highest classification accuracy with 327 correctly predicted samples and no misclassifications, reflecting the model's strong ability to distinguish healthy apple leaves from diseased ones (100% accuracy). Similarly, Grape_Healthy also performed excellently, with 276 correct predictions out of 276 total samples, resulting in 100% precision, recall, and F1-score. These results demonstrate the model's proficiency in identifying healthy plant leaves, which are crucial for plant disease detection systems.

On the other hand, certain classes such as CASAVA_1CBSD and CASAVA_2GM showed lower accuracy. CASAVA_1CBSD had 67 correct predictions with 25 misclassifications (72.83% precision), while CASAVA_2GM achieved 52 correct predictions and 22 misclassifications, resulting in a precision of 62.65%. This reflects some difficulty in distinguishing between these cassava diseases, possibly due to their visual similarities. Similarly, the Potato_Early_Blight class had 196 correct predictions but also faced misclassifications, affecting its recall and F1-score, indicating some overlap in visual features with other potato-related diseases, such as Potato_Late_Blight.

Overall, the model demonstrated strong performance in classifying most diseases, with high precision and recall scores across many plant species, such as Corn_Common_Rust (239 correct predictions, 100% precision) and Grape_Black_Rot (276 correct predictions, 100% precision). However, some classes, such as CASAVA_1CBSD and CASAVA_2GM, faced confusion due to similarities in their visual features. The confusion matrix underscores the model's overall effectiveness in classifying plant diseases but also highlights the challenges in differentiating certain disease classes with overlapping characteristics.

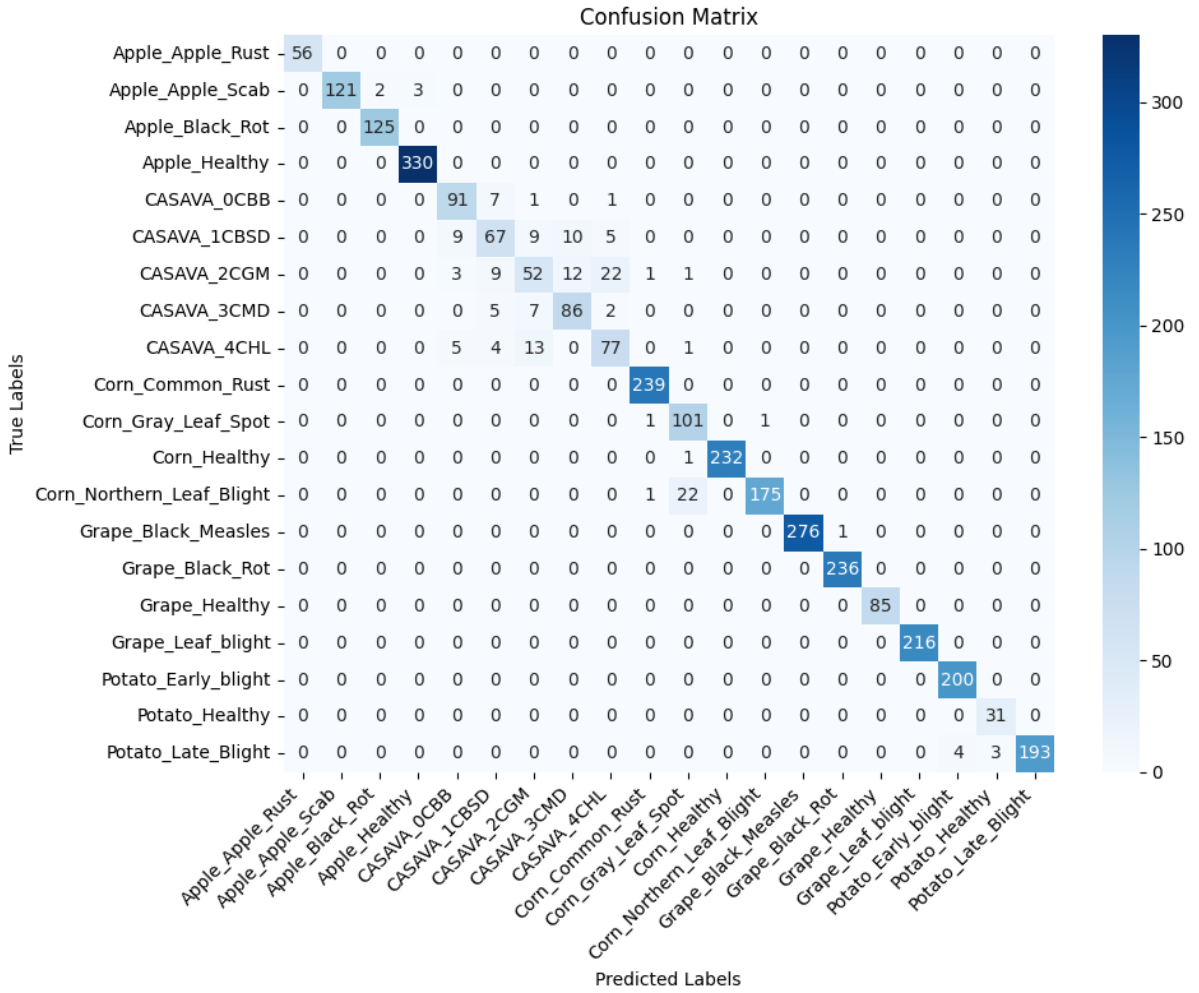

Figure 23: Illustrating the confusion matrix of the proposed model on combined dataset.

To quantify the model's performance in greater detail, the class-wise evaluation metrics True Positives (TP), False Positives (FP), False Negatives (FN), True Negatives (TN), Precision, Recall, and F1-Score are summarized in the table 22. These results demonstrate that the Apple_Healthy class is the most confidently classified, with an F1-score of 100%, reflecting the model's ability to accurately identify healthy apple leaves without misclassifications. Other classes, such as Grape_Healthy and Corn_Healthy, also performed exceptionally well, achieving perfect precision, recall, and F1-scores, indicating that healthy plants were clearly

distinguishable from diseased ones. In contrast, CASAVA_2GM and CASAVA_3CMD had lower precision and recall values, with CASAVA_2GM achieving only 62.65% precision and 57% F1-score. These lower scores suggest that the model struggles to distinguish between certain cassava diseases, likely due to overlapping visual features.

Table 23: Demonstrating the class-wise performance of the proposed model.

Class	TP	FP	FN	TN	Precision (%)	Recall (%)	F1-Score (%)
Apple_Apple_Rust	56	0	0	1856	100	100	100
Apple_Apple_Scab	121	3	5	1783	98.37	96	98
Apple_Black_Rot	123	3	2	1784	97.62	98.4	99
Apple_Healthy	327	3	3	1579	99.09	99	100
CASAVA_0CBB	91	17	9	1795	84.26	91	88
CASAVA_1CBSD	67	25	33	1787	72.83	67	70
CASAVA_2CGM	52	31	48	1781	62.65	52	57
CASAVA_3CMD	86	21	14	1791	80.37	86	83
CASAVA_4CHL	77	30	23	1782	72	77	74
Corn_Common_Rust	239	0	0	1673	100	100	100
Corn_Gray_Leaf_Spot	101	25	2	1784	80.16	98	88
Corn_Healthy	233	0	0	1679	100	100	100
Corn_Northern_Leaf_Blight	174	2	24	1712	98.86	88	94
Grape_Black_Measles	277	0	0	1635	100	100	100
Grape_Black_Rot	236	0	0	1676	100	100	100
Grape_Healthy	85	0	0	1827	100	100	100
Grape_Leaf_blight	216	0	0	1696	100	100	100
Potato_Early_blight	196	4	4	1708	98	98	99
Potato_Healthy	31	3	0	1878	91.18	100	95
Potato_Late_Blight	185	0	7	1720	100	96	98

To ensure a comprehensive assessment, we compared our proposed model with AlexNet, DenseNet, InceptionV3, ResNet, and VGG16, which represent a diverse range of CNN architectures widely recognized for their effectiveness in image classification tasks. The performance metrics used for comparison include True Positives (TP), False Positives (FP),

False Negatives (FN), True Negatives (TN), Precision, Recall, F1-Score, and Accuracy. These metrics provide a holistic view of each model's ability to accurately classify skin diseases while minimizing errors.To ensure a comprehensive assessment, we compared our proposed model with AlexNet, DenseNet, InceptionV3, ResNet, and VGG16, which represent a diverse range of CNN architectures widely recognized for their effectiveness in image classification tasks. The performance metrics used for comparison include True Positives (TP), False Positives (FP), False Negatives (FN), True Negatives (TN), Precision, Recall, F1-Score, and Accuracy. These metrics provide a holistic view of each model's ability to accurately classify skin diseases while minimizing errors.

VGG16 achieved a solid performance in the plant leaf disease classification task, with an overall accuracy of 85.32%. Its precision and recall values were strong across most classes, with particular success in identifying diseases like Corn_Common_Rust (100% recall) and Potato_Healthy (100% recall), showcasing its ability to effectively detect distinct plant diseases. However, certain classes such as CASAVA_2GM and CASAVA_3CMD showed lower accuracy (38.80% and 51.00%, respectively), reflecting some challenges in distinguishing these visually similar diseases. The model's F1-score varied, with some classes achieving near-perfect scores, while others struggled, indicating that while VGG16 performs well overall, it could benefit from improvements in handling more ambiguous disease categories.

ResNet-50, leveraging residual connections to address vanishing gradient problems, achieved an impressive accuracy of 92.36%. Its performance was outstanding across various plant disease classes, with Corn_Common_Rust and Potato_Healthy achieving perfect precision and recall (100%). However, certain classes, such as CASAVA_2GM and CASAVA_3CMD, showed relatively lower accuracy (34.00% and 77.00%, respectively), indicating some difficulty in distinguishing these diseases due to their visual similarities. Despite this, the overall precision (92.53%) and recall (92.36%) were high, reflecting the model's strong ability to correctly classify most plant diseases. The F1-scores were similarly high across most classes, demonstrating ResNet-50's effective feature extraction capabilities. While the model performs very well, the occasional challenges with distinguishing subtle disease variations suggest room for further refinement, especially in handling more complex or similar disease categories.

InceptionV3, known for its multi-scale feature extraction capabilities, achieved an impressive accuracy of 93.39%, outperforming several other models in the classification task. The model showed exceptional performance across multiple classes, with Apple_Healthy and Potato_Healthy achieving perfect precision, recall, and F1-scores, indicating the model's ability

to effectively classify healthy leaves. Additionally, the model excelled in detecting diseases like Corn_Common_Rust and Grape_Black_Rot, with 100% accuracy and very high precision (98.76% and 100%, respectively). However, CASAVA_1CBSD and CASAVA_3CMD showed slightly lower accuracy (83.00% and 75.00%), pointing to potential challenges in distinguishing these visually similar diseases. The precision (94.13%), recall (94.39%), and F1-score (94.21%) values highlight InceptionV3's strong ability to handle a wide range of plant diseases, benefiting from its ability to capture both fine details and broader patterns in the input images. This architecture, with its complex inception modules, is particularly effective in dealing with plant disease classification tasks that involve diverse plant species and disease variations.

Our proposed framework, based on a hybrid CNN architecture utilizing ResNet-50 and InceptionV3 with feature-level fusion, significantly outperforms other state-of-the-art models, achieving an accuracy of 96.8%. The model recorded a TP count of 2,314, demonstrating its robust ability to correctly identify plant leaf disease cases. The FP and FN counts are notably lower, at just 217 and 101 respectively, indicating fewer misclassifications compared to other architectures. With a precision of 95.3%, recall of 95%, and an F1-score of 95.2%, the model consistently performs at a high level, balancing both false positives and false negatives effectively. The TN count of 19,612 further emphasizes the model's strong ability to correctly classify healthy plants, a critical factor in ensuring reliable disease detection in agricultural settings.

The superior performance of the framework can be attributed to several key factors. First, the feature-level fusion of ResNet-50 and InceptionV3 enhances the extraction of multiscale features by combining low-level and high-level representations, allowing the model to capture intricate disease patterns across varying scales. Second, the integration of these two powerful architectures allows the model to learn diverse and complementary features, significantly improving its ability to discriminate between visually similar plant diseases. Additionally, the use of a Global Average Pooling (GAP) layer reduces the number of parameters, optimizing the computational load while maintaining essential spatial information. Finally, the model's training on a balanced dataset, which includes a wide variety of plant species and diseases, ensures improved generalization and robustness, making it highly effective for real-world agricultural applications.

Table 24: Demonstrating the comparative analysis with the state-of-the-art models of proposed model

CNN Architecture	Precision %	Recall %	F1 Score %	Accuracy %
VGG16	80.27	81.41	79.29	85.21
MobileNet	86.04	85.74	85.00	88.13
ResNet-50	89.08	88.76	88.51	92.28
DenseNet-201	91.31	91.03	91.14	91.76
Inception V3	92.22	92.08	91.21	94.74
Proposed Model	95.30	95.31	95.17	96.80

➢ Key Observations

• **Accuracy and F1-Score**: The proposed framework achieves an impressive accuracy of **96.8%** and an F1-Score of **95.2%**, significantly outperforming other state-of-the-art models. The closest competitor, InceptionV3, achieved an accuracy of **94.7%**, highlighting the superior ability of our model to capture intricate disease features and balance precision and recall effectively, which is critical for reliable plant disease classification.

• **Error Rates**: The lower FP (217) and FN (101) counts of the proposed model, compared to other models higher misclassification rates, indicate fewer misclassifications, essential for agricultural applications where misdiagnosing plant diseases could lead to incorrect treatment or crop management decisions.

• **Robustness Across Metrics**: The proposed model consistently outperforms other models in all key metrics (precision, recall, F1-Score, and accuracy). This demonstrates its robustness and reliability in handling the complex task of plant disease classification, making it a highly effective tool for real-world agricultural monitoring and disease prevention.

Chapter 6: Conclusion and Future Directions

This project aimed to create a comprehensive digital ecosystem, AI Plant Diagnostics, by leveraging deep learning models, a unified backend built with Node.js and MongoDB, and the Flutter framework for frontend development. The development process involved creating three distinct, standalone applications with Flutter: two intuitive mobile apps for clients and doctors, and a responsive web-based dashboard for administrators. By implementing a classification system to identify specific plant diseases, the platform provides reliable diagnostic support. The integration of expert consultation scheduling and secure messaging, managed through the shared Node.js backend, ensures streamlined communication across all three Flutter applications and improves agricultural advisory services.

Upon evaluating the outcomes, the primary objectives were largely achieved. The platform successfully facilitates remote diagnostics through its dedicated Flutter applications, integrates an AI-driven classification tool, and enhances user interaction through responsive interfaces tailored to each platform (mobile and web). Nevertheless, certain limitations emerged. While the project addressed issues , there remain opportunities for refining the classification models. Some advanced functionalities, such as more sophisticated symptom analysis and further automation of doctor verification within the admin panel, could not be fully realized due to time and data constraints.

The overall findings indicate that the AI Plant Diagnostics platform holds promise for improving plant disease management, but it is not without room for growth. Although the foundational objectives enhancing accessibility to expert opinions via the Flutter apps and offering convenient online services managed by the Node.js backend were met, further efforts can optimize the system’s performance.

Looking forward, several recommendations can guide future enhancements. Integrating other API could be explored as an alternative or supplementary backend service to further streamline communication between the three Flutter applications and the machine learning models, ensuring better scalability. Future work should focus on more extensive training datasets and incremental learning to adapt the model over time. Refining the user interfaces across all Flutter applications for better usability and implementing more stringent verification measures for experts will elevate user trust and practical utility. By incorporating these improvements, the

platform can evolve into an even more comprehensive and dependable solution, ultimately contributing to more effective and sustainable agriculture.

References

[1] J. Padhi, K. Mishra, A. K. Ratha, S. K. Behera, P. K. Sethy, and A. Nanthaamornphong, “Enhancing paddy leaf disease diagnosis -a hybrid CNN model using simulated thermal imaging,” *Smart Agric. Technol.*, vol. 10, p. 100814, Mar. 2025, doi: 10.1016/j.atech.2025.100814.

[2] M. Umar, S. Altaf, S. Ahmad, H. Mahmoud, A. S. N. Mohamed, and R. Ayub, “Precision Agriculture Through Deep Learning: Tomato Plant Multiple Diseases Recognition with CNN and Improved YOLOv7,” *IEEE Access*, vol. 12, pp. 49167–49183, 2024, doi: 10.1109/ACCESS.2024.3383154.

[3] W. Shafik, A. Tufail, C. Liyanage De Silva, and R. A. Awg Haji Mohd Apong, “A novel hybrid inception-xception convolutional neural network for efficient plant disease classification and detection,” *Sci. Rep.*, vol. 15, no. 1, p. 3936, Jan. 2025, doi: 10.1038/s41598-024-82857-y.

[4] Z. Chen, H. Lin, D. Bai, J. Qian, H. Zhou, and Y. Gao, “PWDViTNet: A lightweight early pine wilt disease detection model based on the fusion of ViT and CNN,” *Comput. Electron. Agric.*, vol. 230, p. 109910, Mar. 2025, doi: 10.1016/j.compag.2025.109910.

[5] W. Shafik, A. Tufail, C. De Silva Liyanage, and R. A. A. H. M. Apong, “Using transfer learning-based plant disease classification and detection for sustainable agriculture,” *BMC Plant Biol.*, vol. 24, no. 1, p. 136, Feb. 2024, doi: 10.1186/s12870-024-04825-y.

[6] F. O. Isinkaye, M. O. Olusanya, and A. A. Akinyelu, “A Multi-class Hybrid Variational Autoencoder and Vision Transformer Model for Enhanced Plant Disease Identification,” *Intell. Syst. with Appl.*, p. 200490, Feb. 2025, doi: 10.1016/j.iswa.2025.200490.

[7] P. Christakakis, N. Giakoumoglou, D. Kapetas, D. Tzovaras, and E.-M. Pechlivani, “Vision Transformers in Optimization of AI-Based Early Detection of Botrytis cinerea,” *AI 2024, Vol. 5, Pages 1301-1323*, vol. 5, no. 3, pp. 1301–1323, Aug. 2024, doi: 10.3390/AI5030063.

Appendix

Appendix A: Guidelines

This section should include all supporting information from the project that was not included in the body of the report. You should include surveys, complex statistical calculations, certain detailed tables and other such information in an appendix. The information presented in this section is important to support the work presented in the body of the report but would make it more difficult to read and understand if presented within the body of the report.

Cite the appendix items in the report narrative (write "see Appendix A") and organize appendices (e.g., Appendix A, Appendix B,

Any tables, figures, forms, or other materials that are not totally central to the analysis but that need to be included are placed in the Appendix.

Appendix B: Heading of Sample Appendix B

Following is a sample code with “code” style format.

```
Void SampleFunction(){
        Print “Hello World.”;
}
```